Machine Learning

AIML CZG565

M3 : Linear Models for Regression

BITS Pilani
Pilani Campus

Course Faculty of MTech Cluster

BITS – CSIS - WILP

Disclaimer and Acknowledgement

- The content for these slides has been obtained from books and various other source on the Internet
- We here by acknowledge all the contributors for their material and inputs.
- We have provided source information wherever necessary
- Students are requested to refer to the textbook w.r.t detailed content of the presentation deck shared over canvas
- We have reduced the slides from canvas and modified the content flow to suit the requirements of the course and for ease of class presentation

Source: "Probabilistic Machine Learning, An Introduction", Kevin P. Murphy, Slides of Prof.Sugata, Prof. Chetana from BITS Pilani, Prof. Raja vadhana from BITS Pilani , CS109 and CS229 stanford lecture notes and many others who made their course materials freely available online.

Course Plan

M1	Introduction
M2	Machine learning Workflow
M3	Linear Models for Regression
M4	Linear Models for Classification
M5	Decision Tree
M6	Instance Based Learning
M7	Support Vector Machine
M8	Bayesian Learning
M9	Ensemble Learning
M10	Unsupervised Learning
M11	Machine Learning Model Evaluation/Comparison

Agenda

- Linear Model for Regression
- Direct solution vs Iterative Method
- Gradient Descent
- Linear Basis Function
- Notion of Bias vs Variance

Linear Regression

Inductive Learning Hypothesis : Interpretation

- Target Concept
- Discrete : f(x) ∈ {Yes, No, Maybe} Classification
- Continuous : f(x) ∈ [20-100] Regression
- Probability Estimation : f(x) ∈ [0-1]

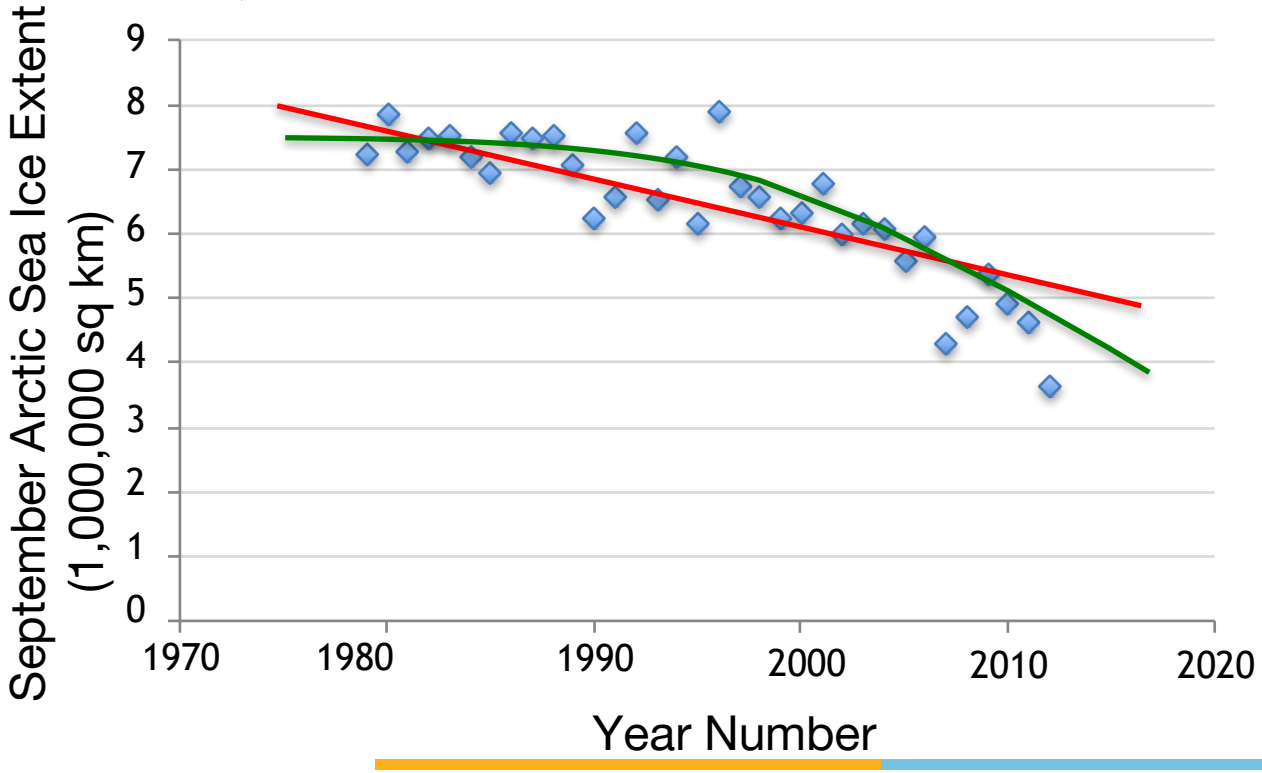
Sky	AirTemp	Altitude	Wind	Water	Forecast	Humidity
Sunny	Warm	Normal	Strong	Warm	Same	60
Sunny	Warm	High	Strong	Warm	Same	75
Rainy	Cold	High	Strong	Warm	Change	70
Sunny	Warm	High	Strong	Cool	Change	45

Formal Representation: Interpretation

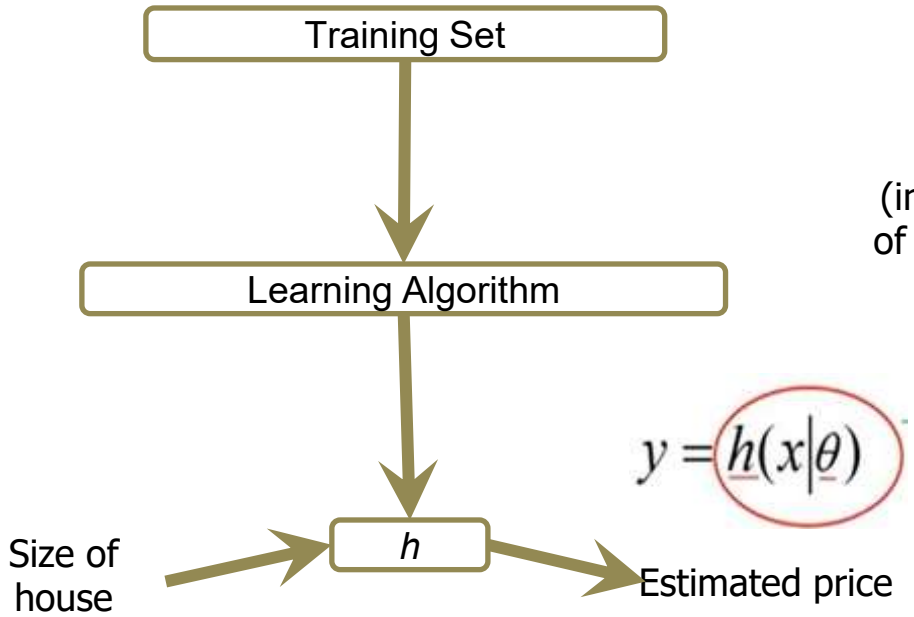
Supervised Learning: Regression

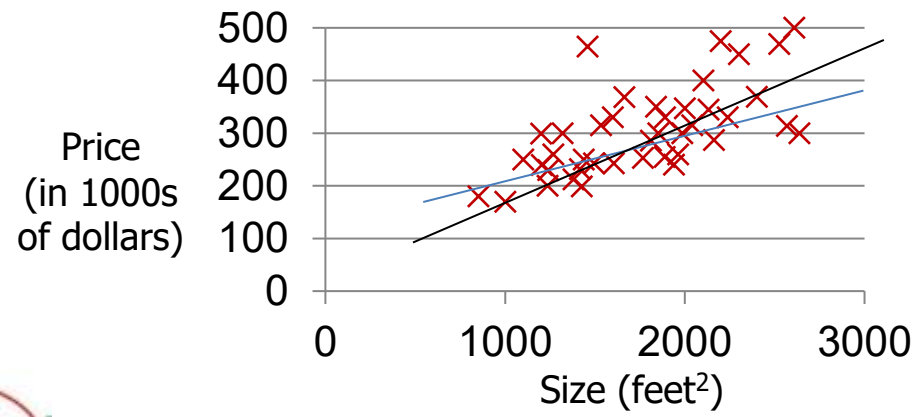
GOAL : Previously unseen records should be assigned a value as accurately as possible.

- Given $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$
- Learn a function $f(x)$ to predict y given x
  - y is real-valued



Machine Learning Process : Interpretation





Size in feet² (x)	Price ($) in 1000's (y)
2104	460
1416	232
1534	315
852	178
…	…

Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$

θ_i's : Parameters

h(.) : Model

How to choose θ_i's ?

Types of Regression Models

Simple Linear regression model

(Education) x ⟶ y (Income)

Multiple Linear regression model

$$h(x) = \sum_{i=0}^{n} \theta_i x_i = \theta^T x,$$

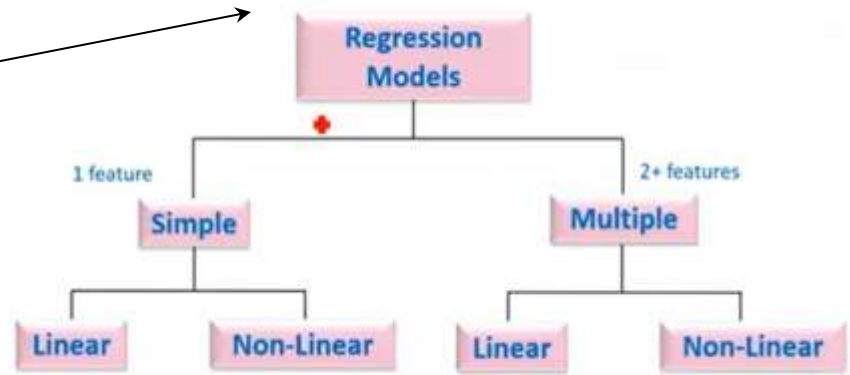
(Education) x_1

(Soft Skills) x_2

(Experience) x_3

(Age) x_4

y (Income)



- Hypothesis:

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_d x_d = \sum_{j=0}^{d} \theta_j x_j$$

Assume $x_0 = 1$

Learning the Model Parameters

Closed Form Solution Approach

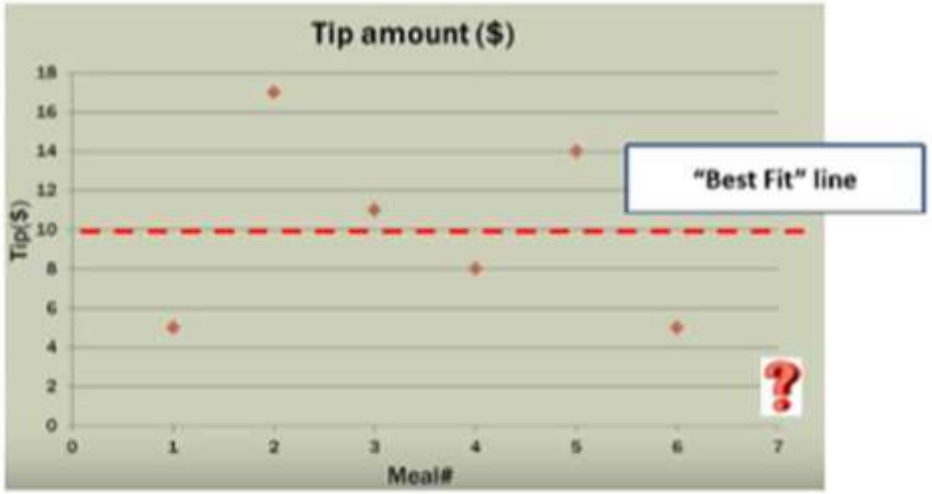
Gradient Descent Approach

How to choose θ_i's ?

Notion of Cost Function

Meal #	Tip amount ($)
1	5.00
2	17.00
3	11.00
4	8.00
5	14.00
6	5.00

mean = $10

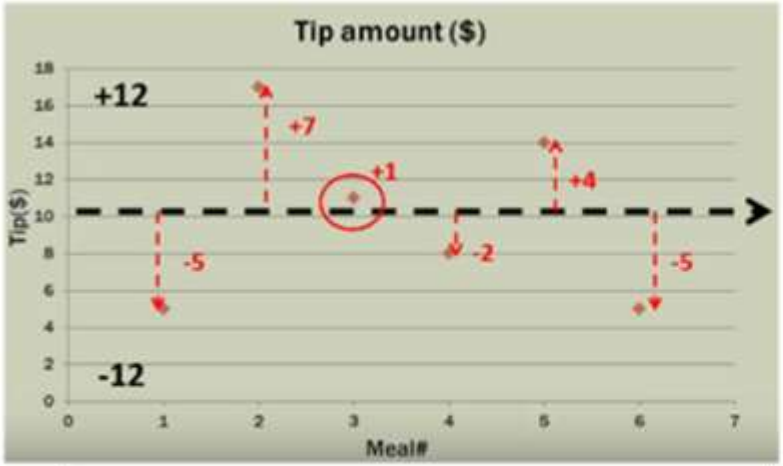


$y = \theta_0$

Notion of Cost Function

Meal #	Tip amount ($)
1	5.00
2	17.00
3	11.00
4	8.00
5	14.00
6	5.00

mean = $10



RESIDULAS (error): Distance from best fit line and the actual values

Meal#	Residual	Residual^2
1	−5	25
2	+7	49
3	+1	1
4	−2	4
5	+4	16
6	−5	25

Sum of Squared Errors (SSE) = 120

Notion of Cost Function

Meal #	Tip amount ($)
1	5.00
2	17.00
3	11.00
4	8.00
5	14.00
6	5.00

mean = $10

Meal	Total bill ($)	Tip amount ($)	$\hat{y}_i = 0.1462x - 0.8188$	$\hat{y}_i$ (predicted tip amount)
	x	y		
1	34	5	$\hat{y}_i = 0.1462(34) - 0.8188$	4.1505
2	108	17	$\hat{y}_i = 0.1462(108) - 0.8188$	14.9693
3	64	11	$\hat{y}_i = 0.1462(64) - 0.8188$	8.5365
4	88	8	$\hat{y}_i = 0.1462(88) - 0.8188$	12.0453
5	99	14	$\hat{y}_i = 0.1462(99) - 0.8188$	13.6535
6	51	5	$\hat{y}_i = 0.1462(51) - 0.8188$	6.6359
	$\bar{x} = 74$	$\bar{y} = 10$	Observed vs. Predicted	

$$y = \theta_0 + \theta_1 x$$

$$J(\theta) = \frac{1}{2n}\sum_{i=1}^{n}(\hat{y}^{(i)} - y^{(i)})^2$$

$$J(\theta) = \frac{1}{2n}\sum_{i=1}^{n}(h_\theta(x^{(i)}) - y^{(i)})^2$$

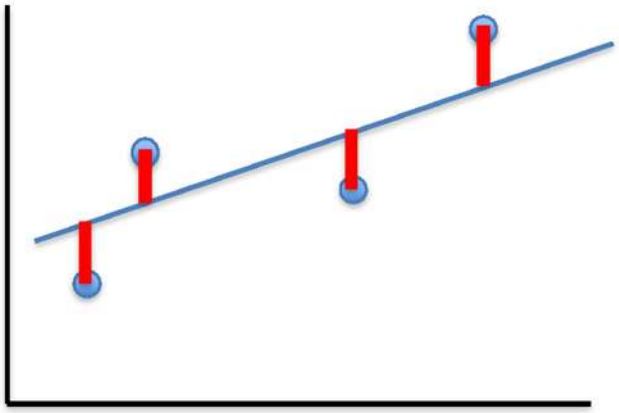
Linear Regression : Least Squares

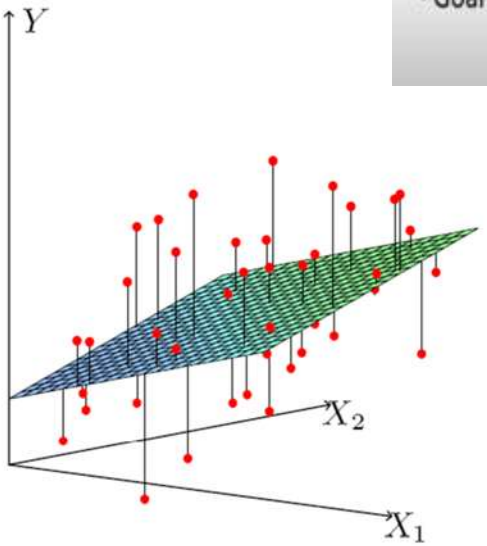
- Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2$$

- Fit by solving $\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$

- Hypothesis $h_\theta(x) = \theta_0 + \theta_1 x$
- Parameters θ_0 and θ_1
- Cost Function $J(\theta_0,\theta_1) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^i) - y^{(i)})^2$
- Goal $\underset{\theta_0,\theta_1}{\text{minimize}}\ J(\theta_0,\theta_1)$





Intuition Behind Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^{n} \left(h_{\boldsymbol{\theta}} \left(\boldsymbol{x}^{(i)} \right) - y^{(i)} \right)^2$$

For insight on J(), let's assume $x \in \mathbb{R}$ so $\boldsymbol{\theta} = [\theta_0, \theta_1]$

Source Credit : Based on example by Andrew Ng

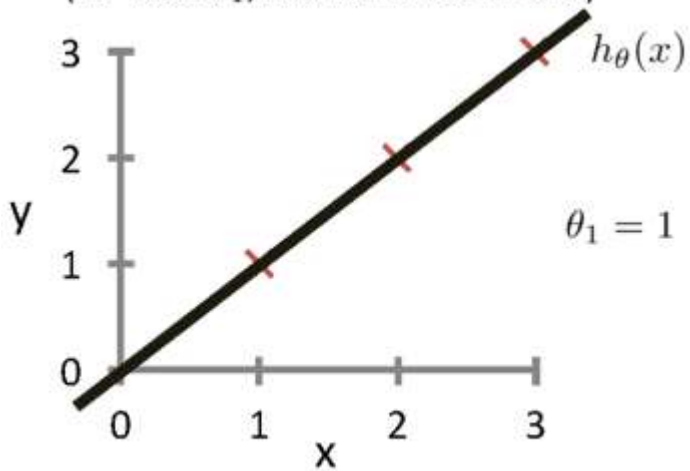
Intuition Behind Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2$$

For insight on J(), let's assume $x \in \mathbb{R}$ so $\boldsymbol{\theta} = [\theta_0, \theta_1]$

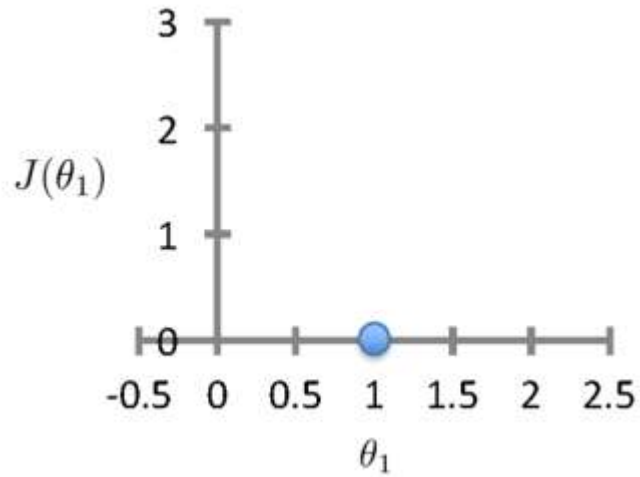
$h_\theta(x)$

(for fixed θ_1, this is a function of x)



$J(\theta_1)$

(function of the parameter θ_1)



Source Credit : Based on example by Andrew Ng

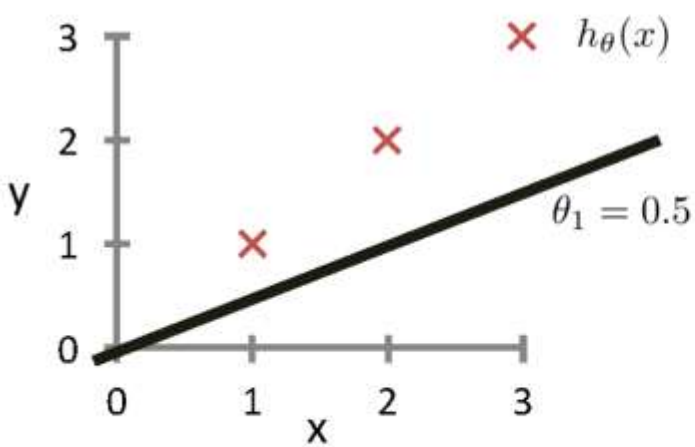
Intuition Behind Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2$$

For insight on J(), let's assume $x \in \mathbb{R}$ so $\boldsymbol{\theta} = [\theta_0, \theta_1]$

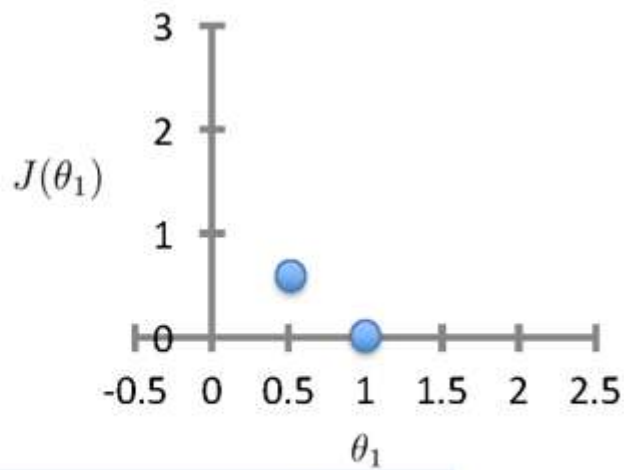
$h_\theta(x)$

(for fixed θ_1, this is a function of x)



$J(\theta_1)$

(function of the parameter θ_1)



Based on example by Andrew Ng

$$J([0, 0.5]) = \frac{1}{2 \times 3}\left[(0.5-1)^2 + (1-2)^2 + (1.5-3)^2\right] \approx 0.58$$

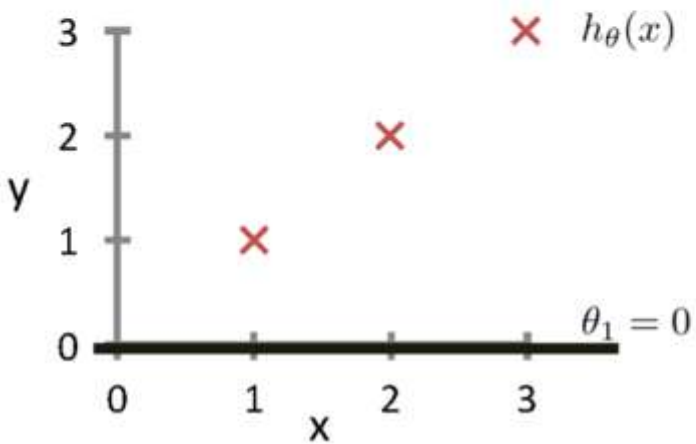
Intuition Behind Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2$$

For insight on J(), let's assume $x \in \mathbb{R}$ so $\boldsymbol{\theta} = [\theta_0, \theta_1]$

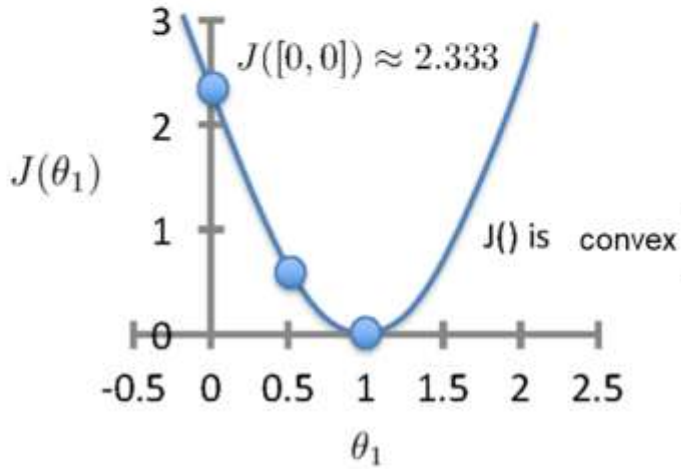
$h_\theta(x)$

(for fixed θ_1, this is a function of x)



$J(\theta_1)$

(function of the parameter θ_1)

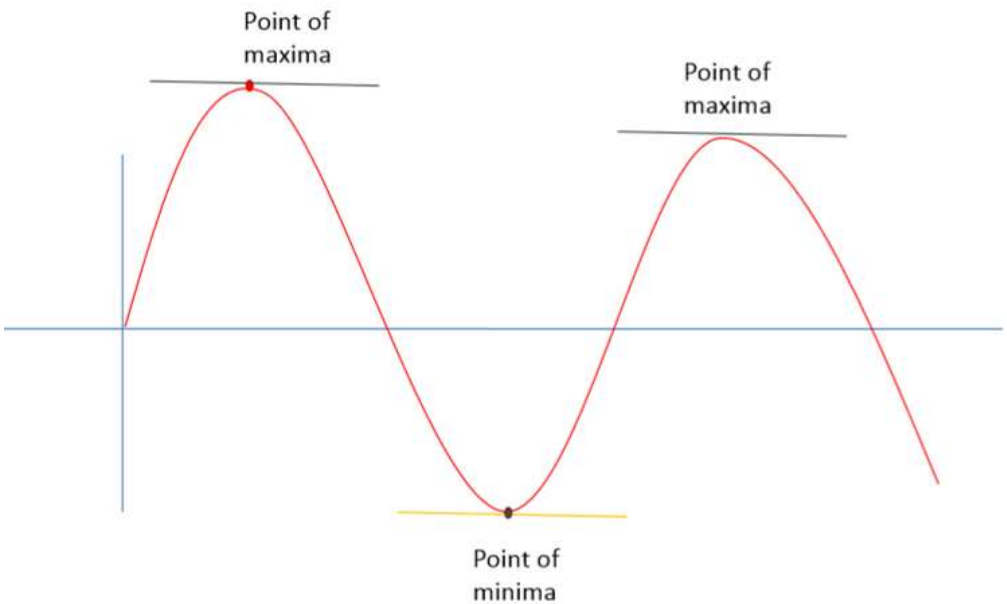


Source Credit : Based on example by Andrew Ng

Notion of Maxima and Minima of a function

In General

- For maxima and minima m=dy/dx=tan 0 ^{0}=0
- dy/dx = 0 means tangent is parralel to X –axis.



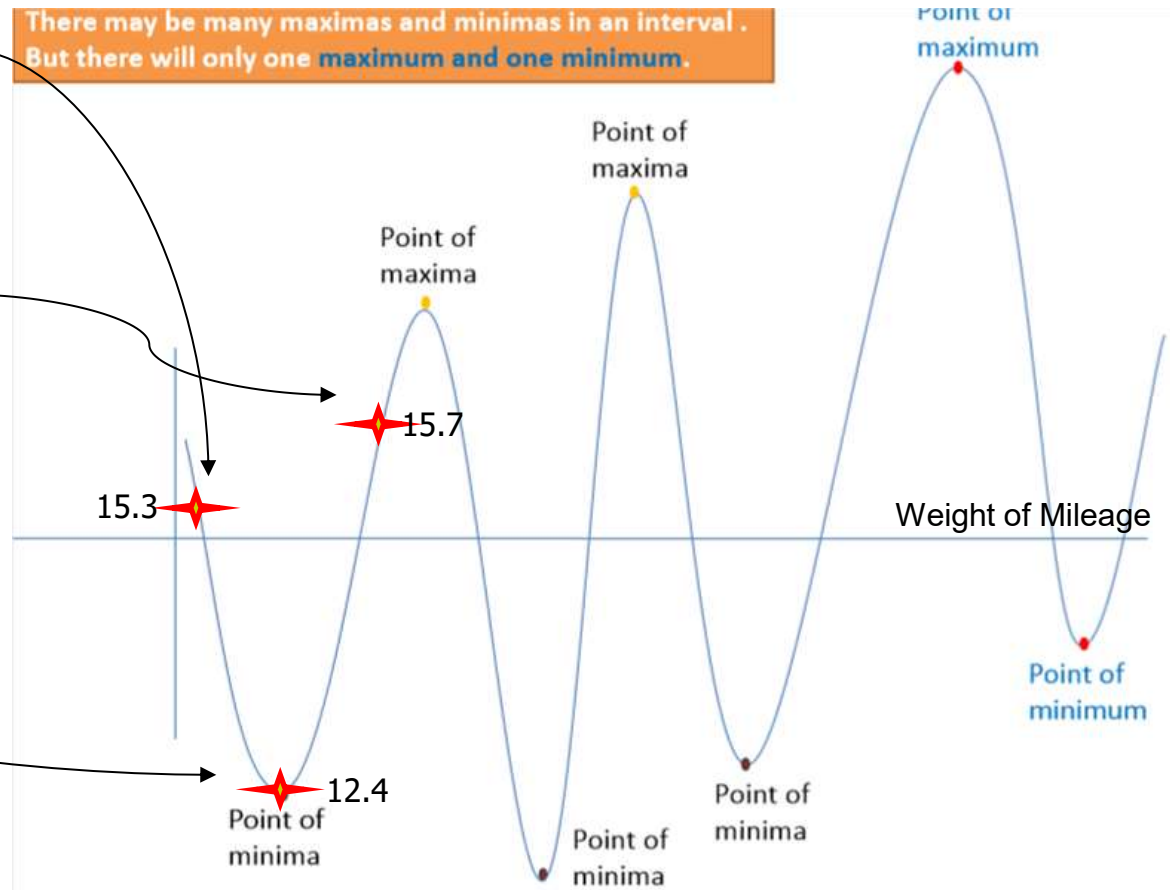
Notion of Maxima and Minima of a function

In Machine Learning

CarPrice = 1 + 0.05Mileage

CarPrice = 1 + 0.25Mileage

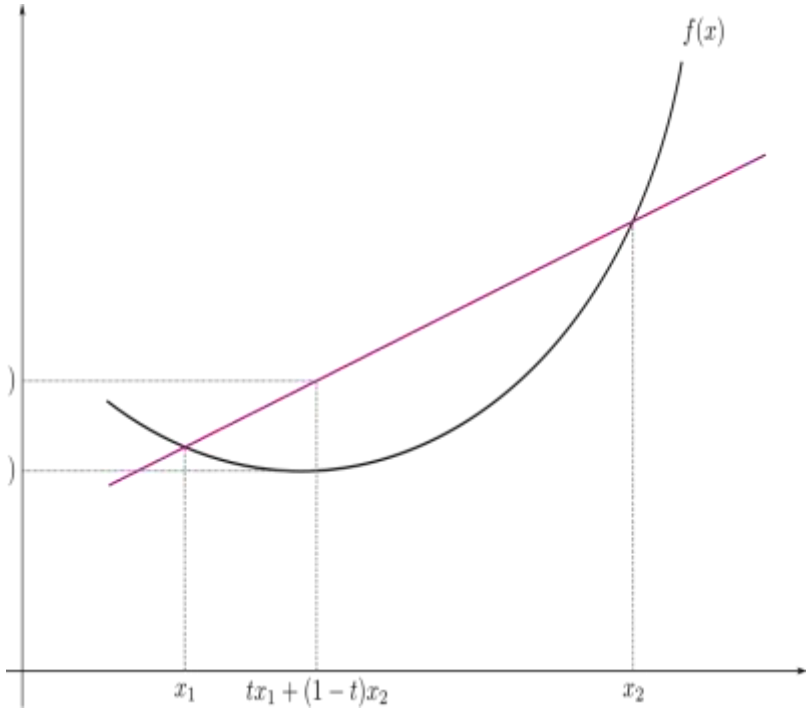
CarPrice = 1 + 0.75Mileage

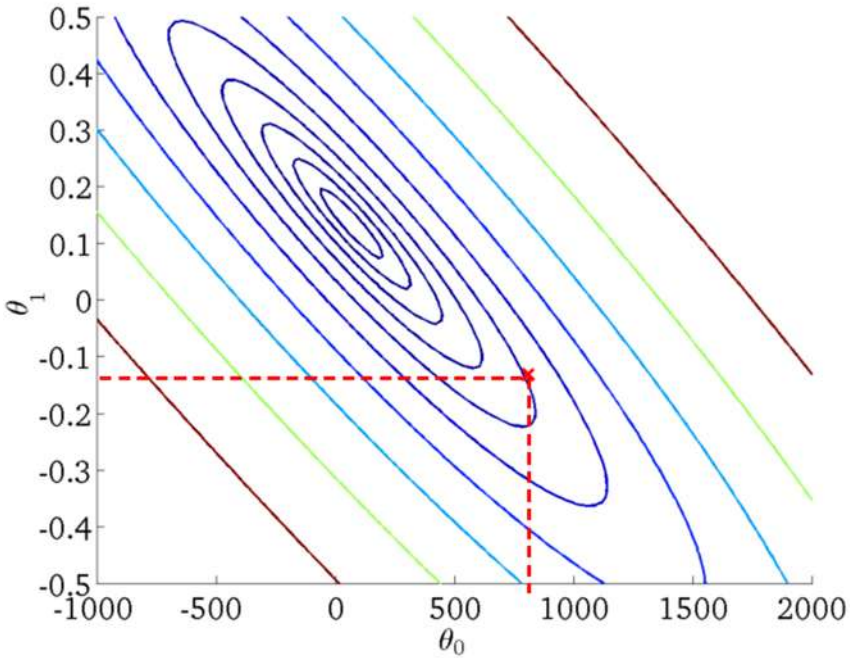


Mileage (in kmpl)	*Car Price (in cr)*
9.8	10.48
9.12	1.75
9.5	6.95
10	2.51

Sum of absolute Error / Training Error

Convex Function : Multivariate View





Real-valued function defined on an *n*-dimensional interval is called **convex** if the line segment between any two points on the graph of the function lies above or on the graph

Closed Form Solution

Vectorization

X No.of.Years of Experience (in Years)	Y Salary Of the Employee (in Lakhs)
1	2
2	3
3	4
4	5
5	6

- Benefits of vectorization
  - More compact equations
  - Faster code (using optimized matrix libraries)
- Consider our model:

$$h(\boldsymbol{x}) = \sum_{j=0}^{d} \theta_j x_j$$

- Let

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix} \qquad \boldsymbol{x}^\intercal = \begin{bmatrix} 1 & x_1 & \dots & x_d \end{bmatrix}$$

- Can write the model in vectorized form as $h(\boldsymbol{x}) = \boldsymbol{\theta}^\intercal \boldsymbol{x}$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} \quad x^T = \begin{bmatrix} 1 & No.of.Years\ Experience \end{bmatrix}$$

Vectorization

X No.of.Years of Experience (in Years)	Y Salary Of the Employee (in Lakhs)
1	2
2	3
3	4
4	5
5	6

- Consider our model for n instances:

$$h\left(\boldsymbol{x}^{(i)}\right) = \sum_{j=0}^{d} \theta_j x_j^{(i)}$$

- Let

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix} \qquad \boldsymbol{X} = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(i)} & \dots & x_d^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \dots & x_d^{(n)} \end{bmatrix}$$

$\mathbb{R}^{(d+1)\times 1}$ $\qquad$ $\mathbb{R}^{n\times(d+1)}$

Let:

$$\boldsymbol{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

- Can write the model in vectorized form as $h_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{X}\boldsymbol{\theta}$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} \quad X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{bmatrix} \quad y = \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

Vectorization

X No.of.Years of Experience (in Years)	Y Salary Of the Employee (in Lakhs)
1	2
2	3
3	4
4	5
5	6

- For the linear regression cost function:

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2$$

$$= \frac{1}{2n}\sum_{i=1}^{n}\left(\boldsymbol{\theta}^{\mathsf{T}}\boldsymbol{x}^{(i)} - y^{(i)}\right)^2$$

$$= \frac{1}{2n}\underbrace{(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y})^{\mathsf{T}}}_{\mathbb{R}^{1\times n}}\,(\underbrace{\boldsymbol{X}}_{\mathbb{R}^{n\times(d+1)}}\underbrace{\boldsymbol{\theta}}_{\mathbb{R}^{(d+1)\times 1}} - \underbrace{\boldsymbol{y}}_{\mathbb{R}^{n\times 1}})$$

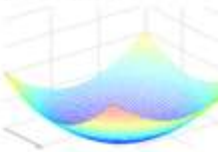
Let:

$$\boldsymbol{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

$$\theta^T = \begin{bmatrix} \theta_0 & \theta_1 \end{bmatrix} \quad X = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} \quad y = \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

Vectorization

- Instead of using GD, solve for optimal $\boldsymbol{\theta}$ analytically
  - Notice that the solution is when $\frac{\partial}{\partial \boldsymbol{\theta}} J(\boldsymbol{\theta}) = 0$



- Derivation:

$$\mathcal{J}(\boldsymbol{\theta}) = \frac{1}{2n}(\boldsymbol{X\theta} - \boldsymbol{y})^\intercal(\boldsymbol{X\theta} - \boldsymbol{y})$$

1 x 1

$$\propto \boldsymbol{\theta}^\intercal \boldsymbol{X}^\intercal \boldsymbol{X}\boldsymbol{\theta} - \boxed{\boldsymbol{y}^\intercal \boldsymbol{X}\boldsymbol{\theta}} - \boxed{\boldsymbol{\theta}^\intercal \boldsymbol{X}^\intercal \boldsymbol{y}} + \boldsymbol{y}^\intercal \boldsymbol{y}$$

$$\propto \boldsymbol{\theta}^\intercal \boldsymbol{X}^\intercal \boldsymbol{X}\boldsymbol{\theta} - 2\boldsymbol{\theta}^\intercal \boldsymbol{X}^\intercal \boldsymbol{y} + \boldsymbol{y}^\intercal \boldsymbol{y}$$

Take derivative and set equal to 0, then solve for $\boldsymbol{\theta}$:

$$\frac{\partial}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}^\intercal \boldsymbol{X}^\intercal \boldsymbol{X}\boldsymbol{\theta} - 2\boldsymbol{\theta}^\intercal \boldsymbol{X}^\intercal \boldsymbol{y} + \cancel{\boldsymbol{y}^\intercal \boldsymbol{y}}) = 0$$

$$(\boldsymbol{X}^\intercal \boldsymbol{X})\boldsymbol{\theta} - \boldsymbol{X}^\intercal \boldsymbol{y} = 0$$

$$(\boldsymbol{X}^\intercal \boldsymbol{X})\boldsymbol{\theta} = \boldsymbol{X}^\intercal \boldsymbol{y}$$

Closed Form Solution:	$\boldsymbol{\theta} = (\boldsymbol{X}^\intercal \boldsymbol{X})^{-1}\boldsymbol{X}^\intercal \boldsymbol{y}$

Vectorization

- Can obtain $\boldsymbol{\theta}$ by simply plugging $\boldsymbol{X}$ and $\boldsymbol{y}$ into

$$\boldsymbol{\theta} = (\boldsymbol{X}^\intercal \boldsymbol{X})^{-1} \boldsymbol{X}^\intercal \boldsymbol{y}$$

$$\boldsymbol{X} = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(i)} & \dots & x_d^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \dots & x_d^{(n)} \end{bmatrix} \qquad \boldsymbol{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

- If $\boldsymbol{X}^\intercal \boldsymbol{X}$ is not invertible (i.e., singular), may need to:
  - Use pseudo-inverse instead of the inverse
    - In python, `numpy.linalg.pinv(a)`
  - Remove redundant (not linearly independent) features
  - Remove extra features to ensure that $d \leq n$

Fit the Linear Regression Model :

Using Closed Form – Problem Type 1

X No.of.Years of Experience (in Years)	Y Salary Of the Employee (in Lakhs)
1	2
2	3
3	4
4	5
5	6

$$\left(X^T \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} * X \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{bmatrix} \right)^{-1} * X^T \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} * y \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

$$\left(\begin{bmatrix} 5 & 15 \\ 15 & 55 \end{bmatrix} \right)^{-1} * X^T \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} * y \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

$$\left(\begin{bmatrix} 1.1 & -0.3 \\ -0.3 & 0.1 \end{bmatrix} \right) * X^T \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} * y \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

$$\begin{bmatrix} 0.8 & 0.5 & 0.2 & -0.1 & -0.4 \\ -0.2 & -0.1 & 0 & 0.1 & -0.2 \end{bmatrix} * y \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

$$\theta = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Can obtain $\boldsymbol{\theta}$ by simply plugging $\boldsymbol{X}$ and $\boldsymbol{y}$ into

$$\boldsymbol{\theta} = (\boldsymbol{X}^\intercal \boldsymbol{X})^{-1} \boldsymbol{X}^\intercal \boldsymbol{y}$$

$$\theta^T = \begin{bmatrix} \theta_0 & \theta_1 \end{bmatrix} \quad X = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} \quad y = \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

Gradient Descent Approach

BITS Pilani

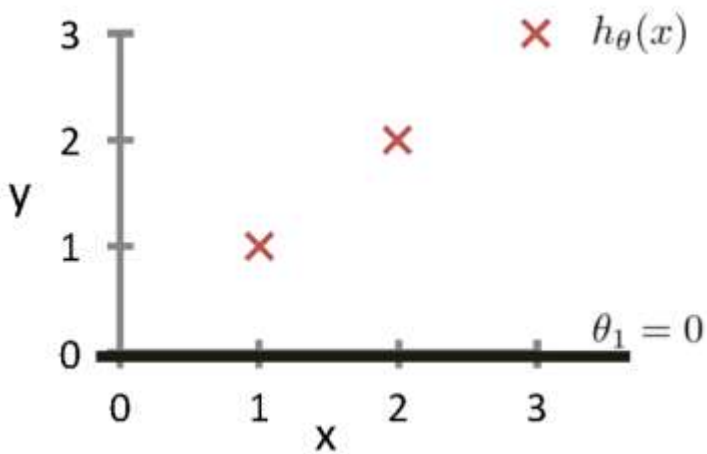
Intuition Behind Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2$$

For insight on J(), let's assume $x \in \mathbb{R}$ so $\boldsymbol{\theta} = [\theta_0, \theta_1]$

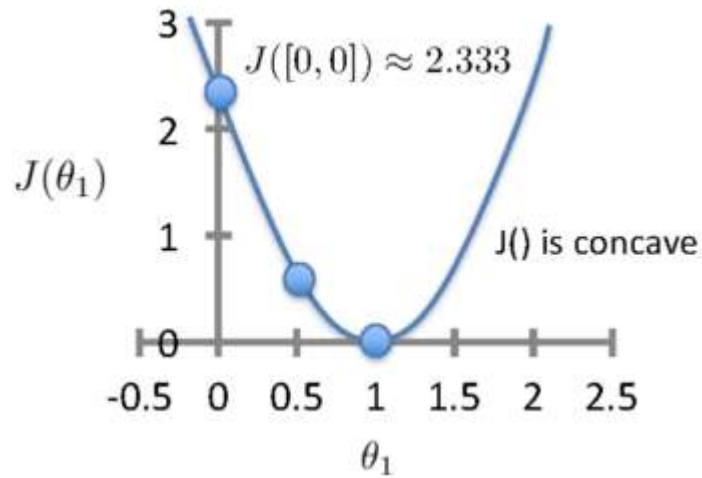
$h_\theta(x)$

(for fixed θ_1, this is a function of x)



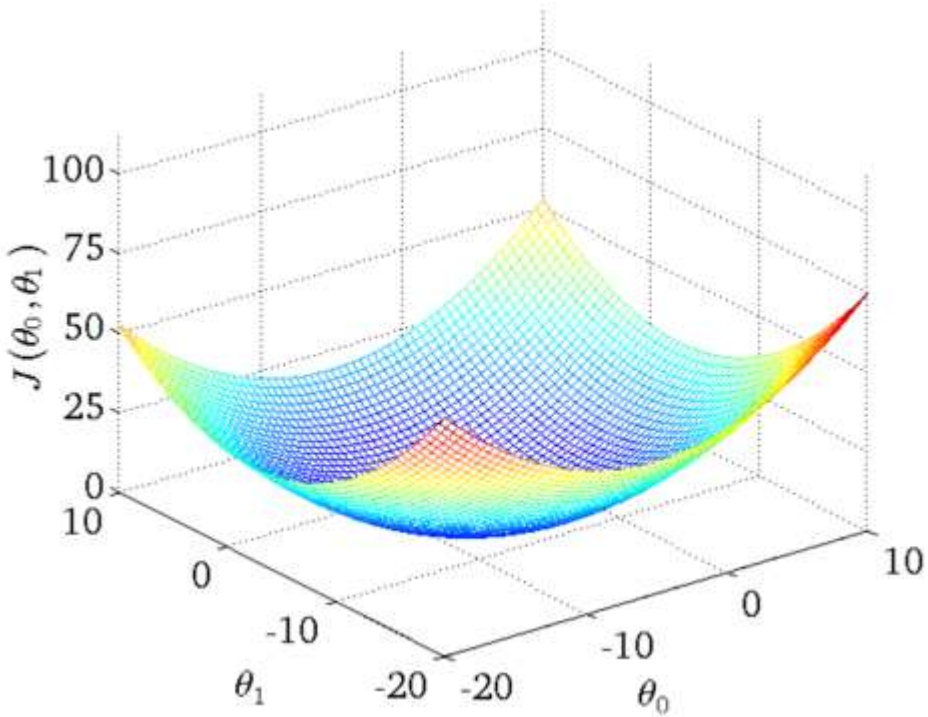
$J(\theta_1)$

(function of the parameter θ_1)



Source Credit : Based on example by Andrew Ng

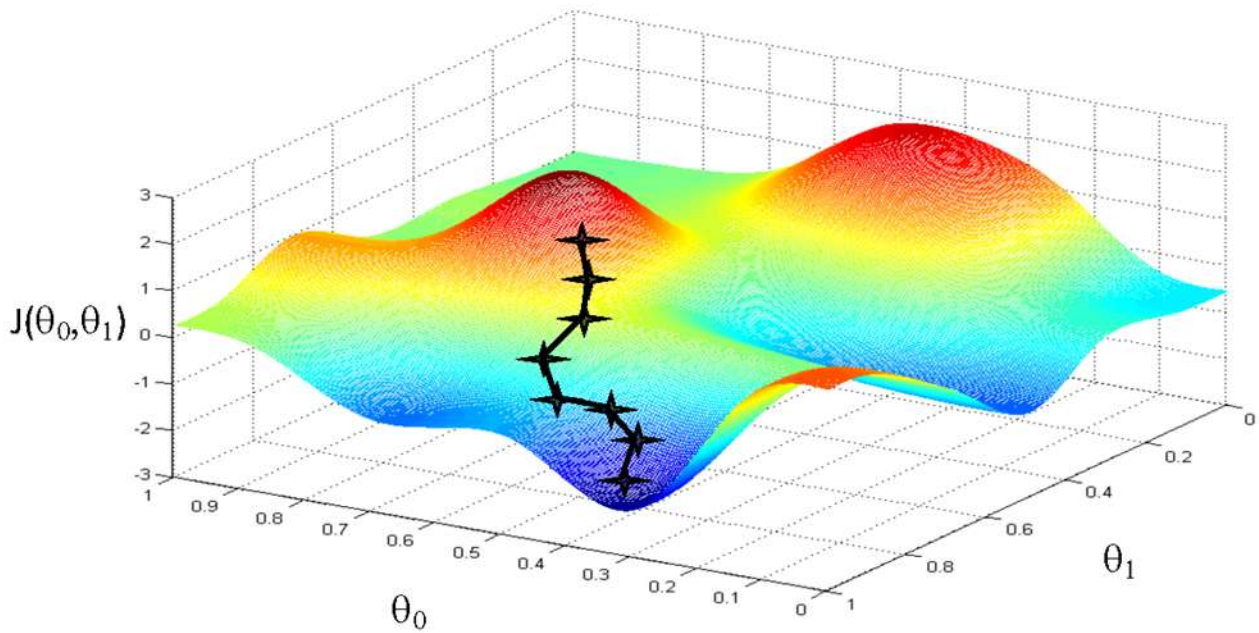
Intuition Behind Cost Function



Source Credit : Based on example by Andrew Ng

Basic Search Procedure

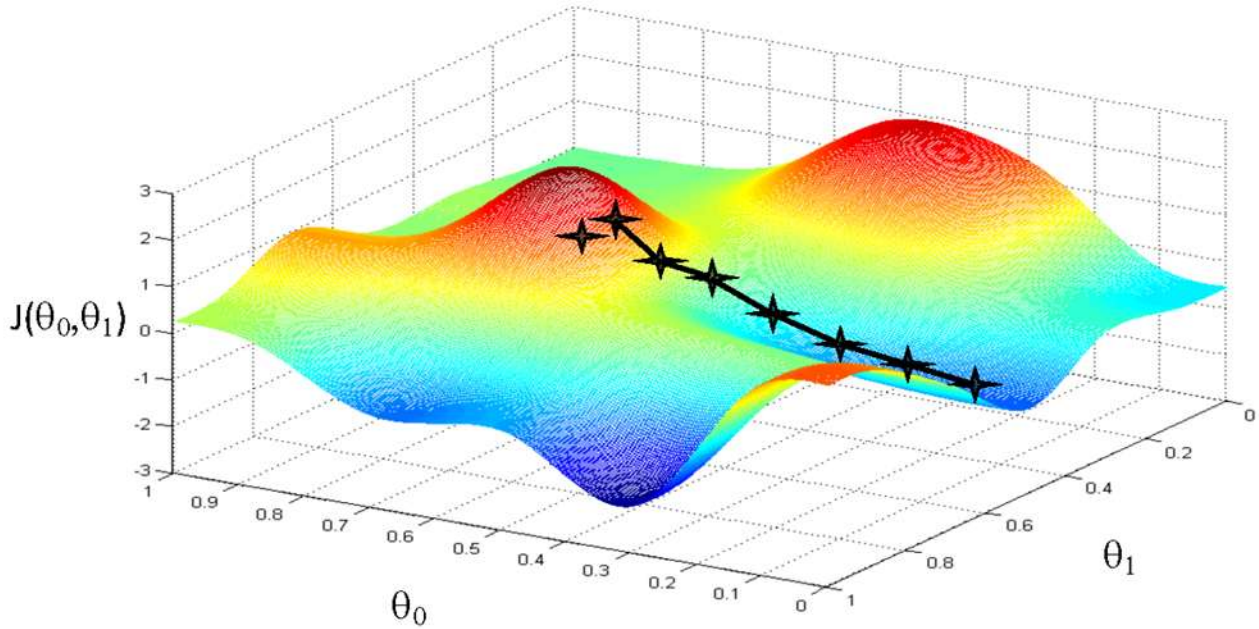
- Choose initial value for $\boldsymbol{\theta}$
- Until we reach a minimum:
  - Choose a new value for $\boldsymbol{\theta}$ to reduce $J(\boldsymbol{\theta})$



Source Credit : Figure by Andrew Ng

Basic Search Procedure

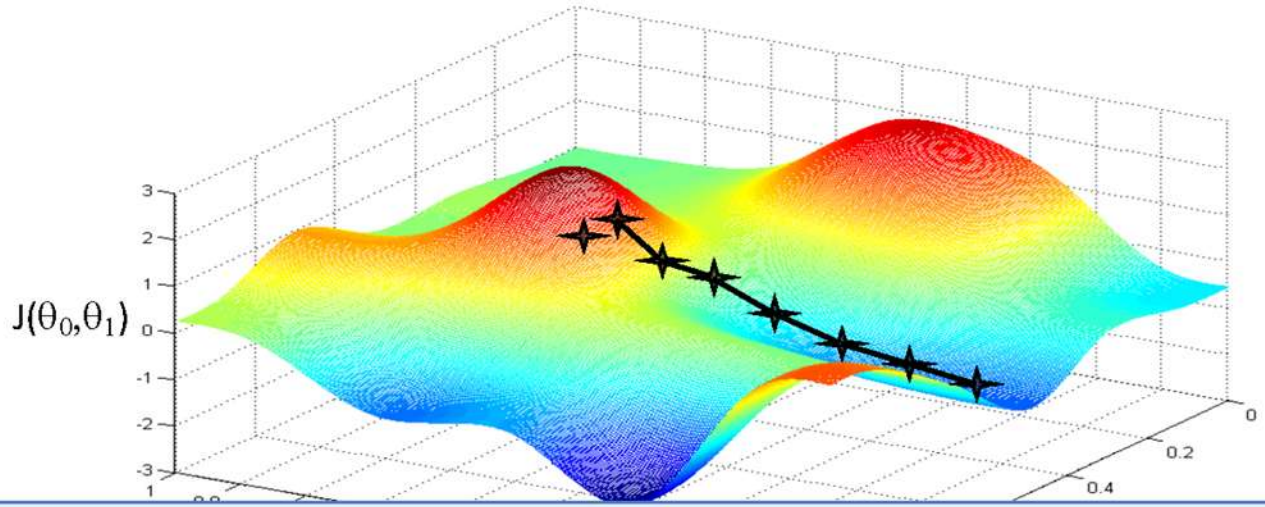
- Choose initial value for $\boldsymbol{\theta}$
- Until we reach a minimum:
  - Choose a new value for $\boldsymbol{\theta}$ to reduce $J(\boldsymbol{\theta})$



Source Credit : Figure by Andrew Ng

Basic Search Procedure

- Choose initial value for $\boldsymbol{\theta}$
- Until we reach a minimum:
  - Choose a new value for $\boldsymbol{\theta}$ to reduce $J(\boldsymbol{\theta})$



Since the least squares objective function is convex (concave), we don't need to worry about local minima

Source Credit : Figure by Andrew Ng

Gradient Descent

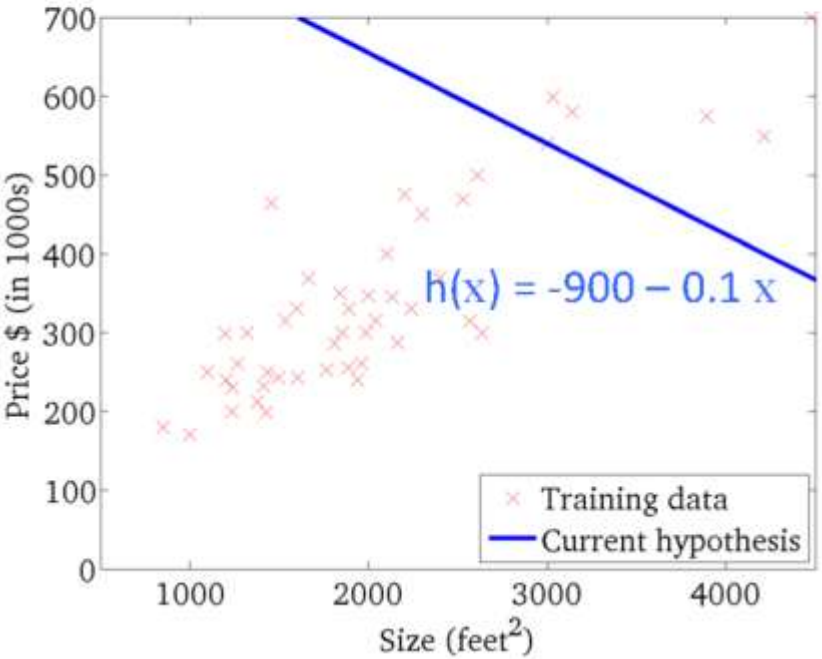
Size in feet^2 (x)	Price ($) in 1000's (y)
2104	460
1416	232
1534	315
852	178
…	…

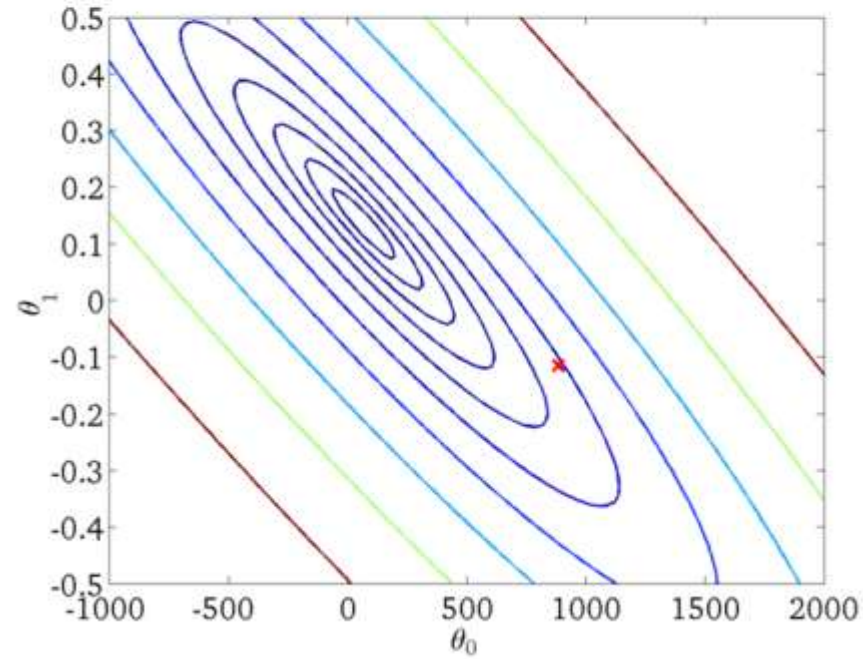
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)

$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



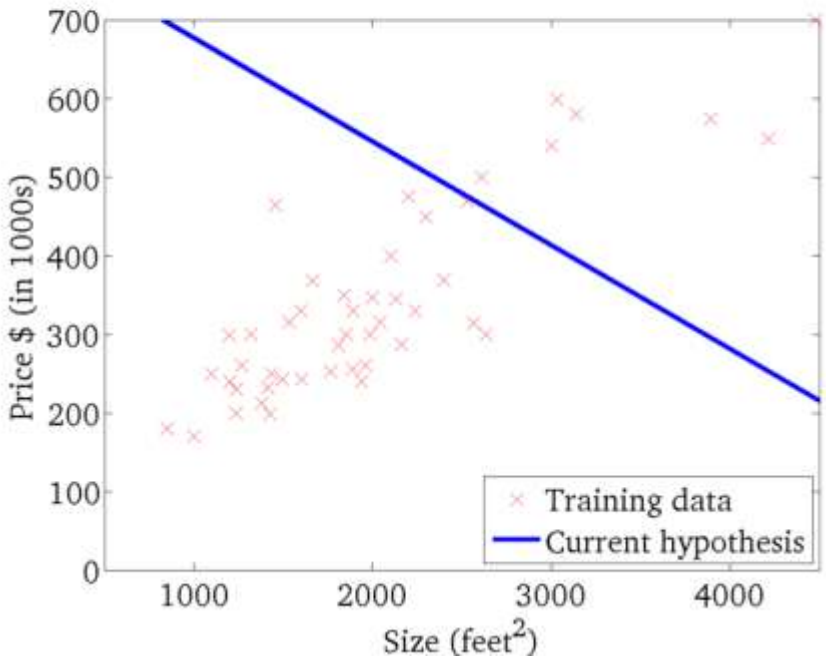


Source Credit : Slide by Andrew Ng

Gradient Descent

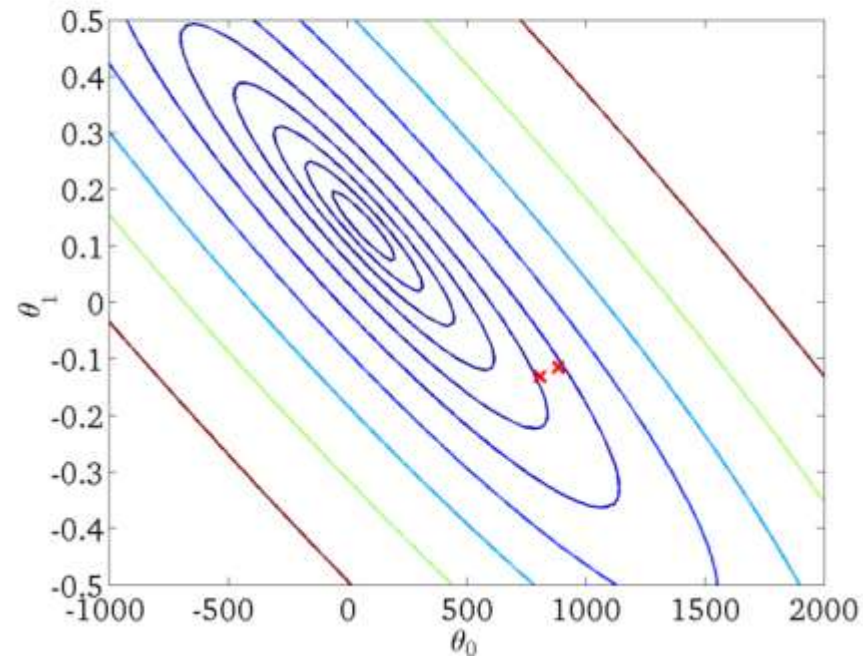
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

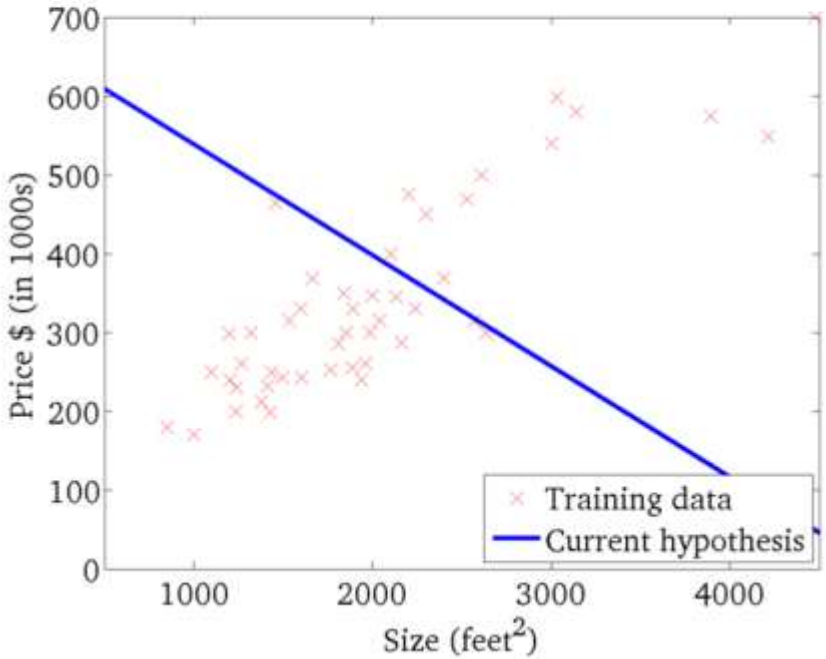


Source Credit : Slide by Andrew Ng

Gradient Descent

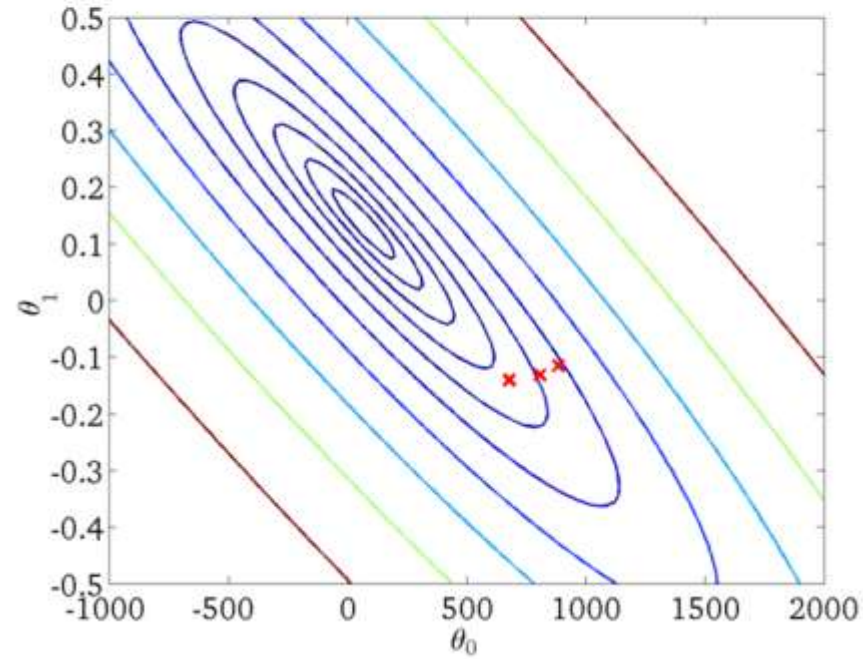
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

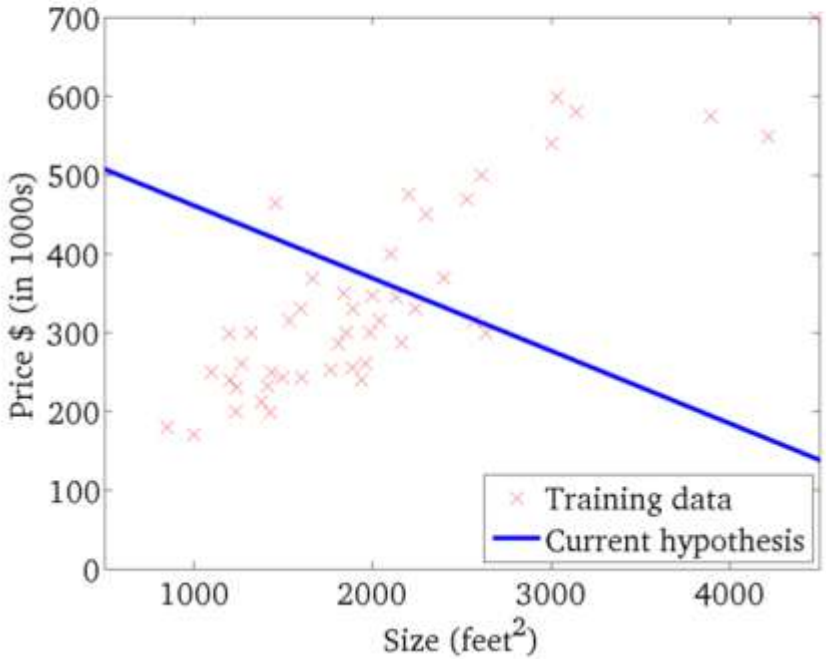


Source Credit : Slide by Andrew Ng

Gradient Descent

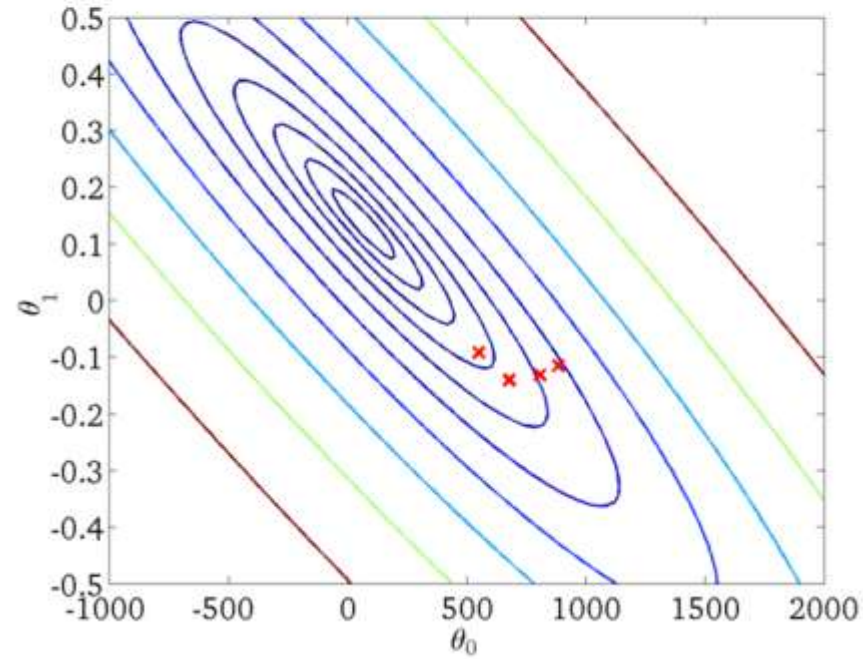
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

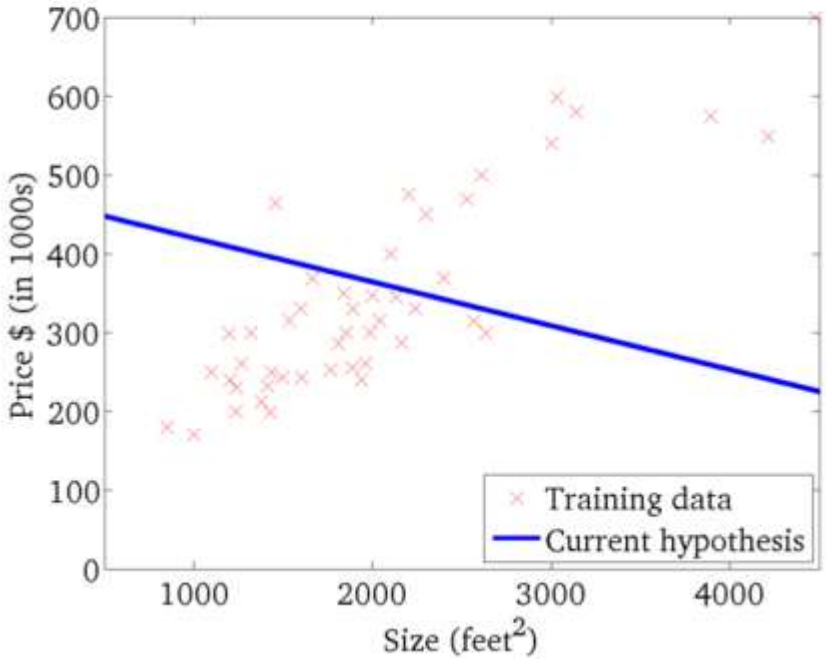


Source Credit : Slide by Andrew Ng

Gradient Descent

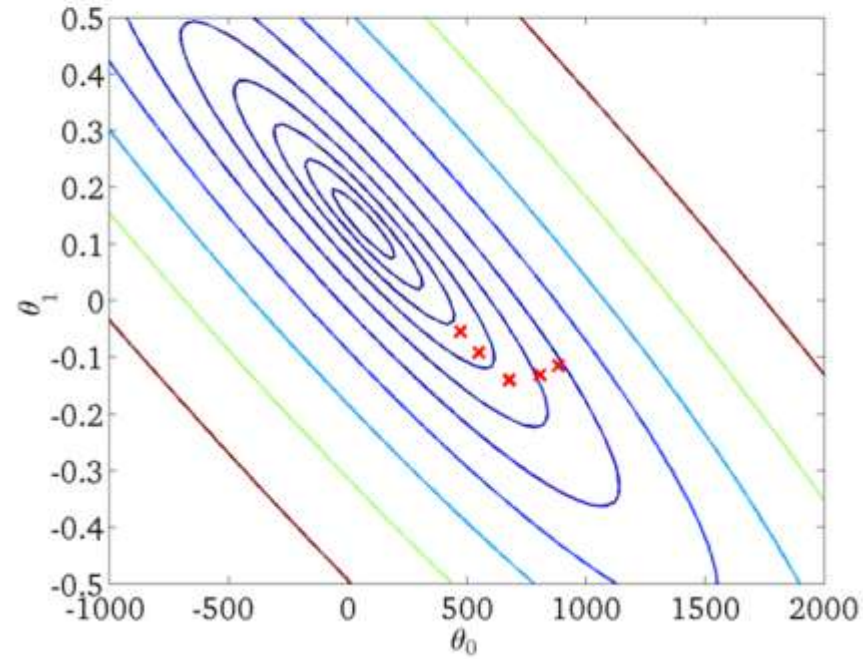
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

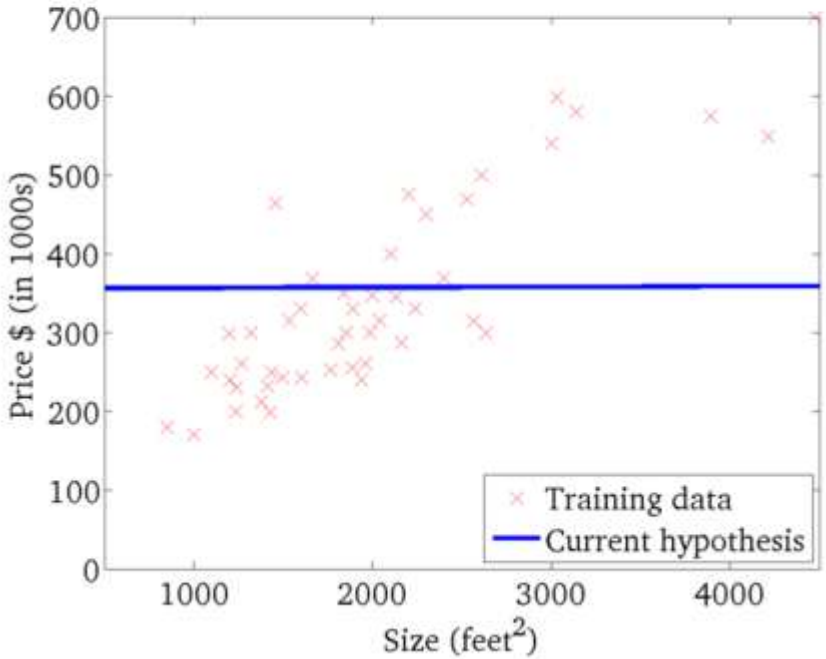


Source Credit : Slide by Andrew Ng

Gradient Descent

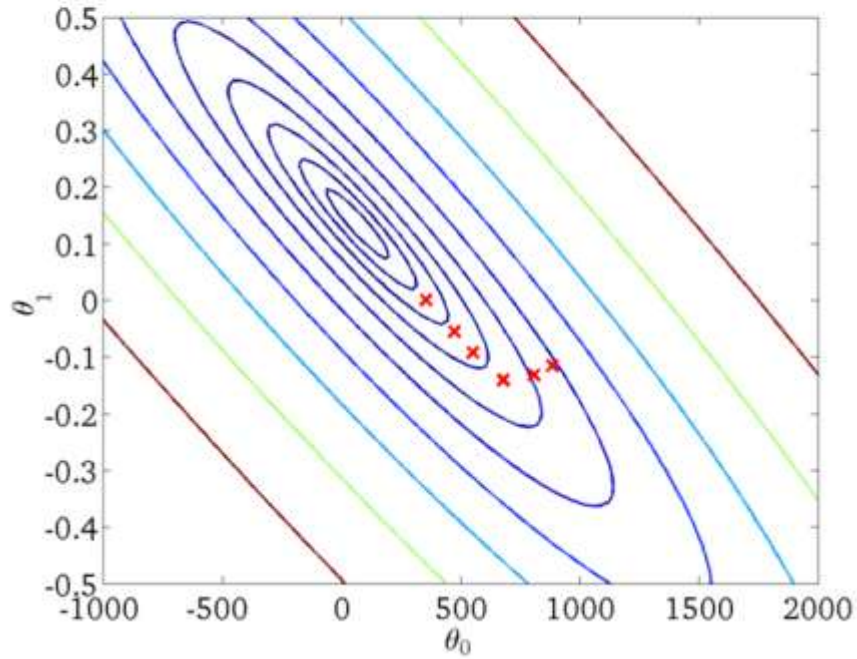
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

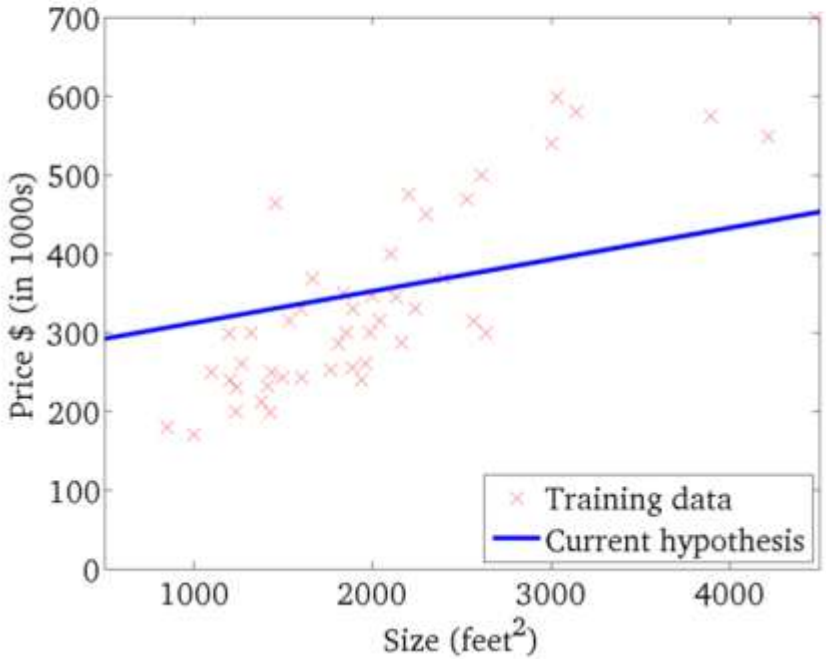


Source Credit : Slide by Andrew Ng

Gradient Descent

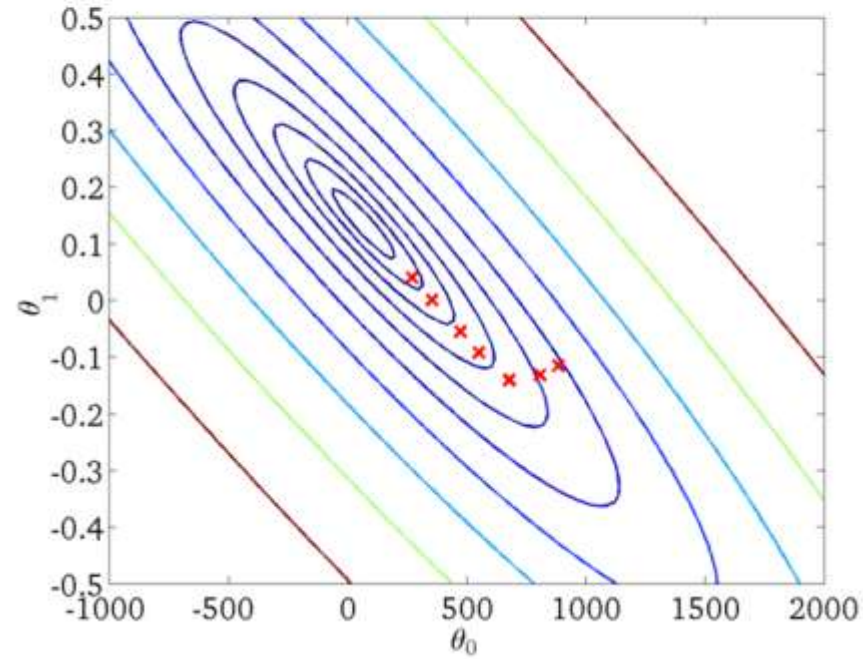
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

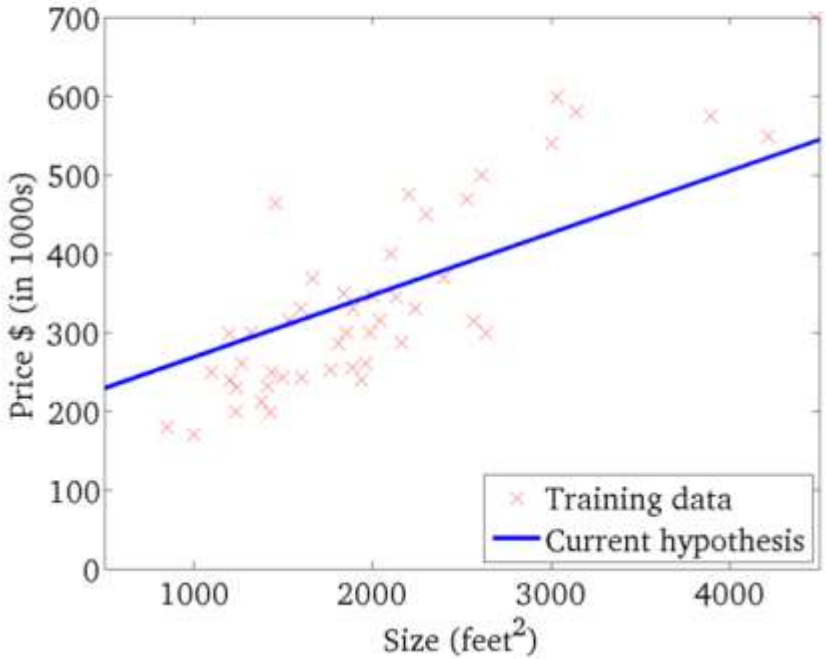


Source Credit : Slide by Andrew Ng

Gradient Descent

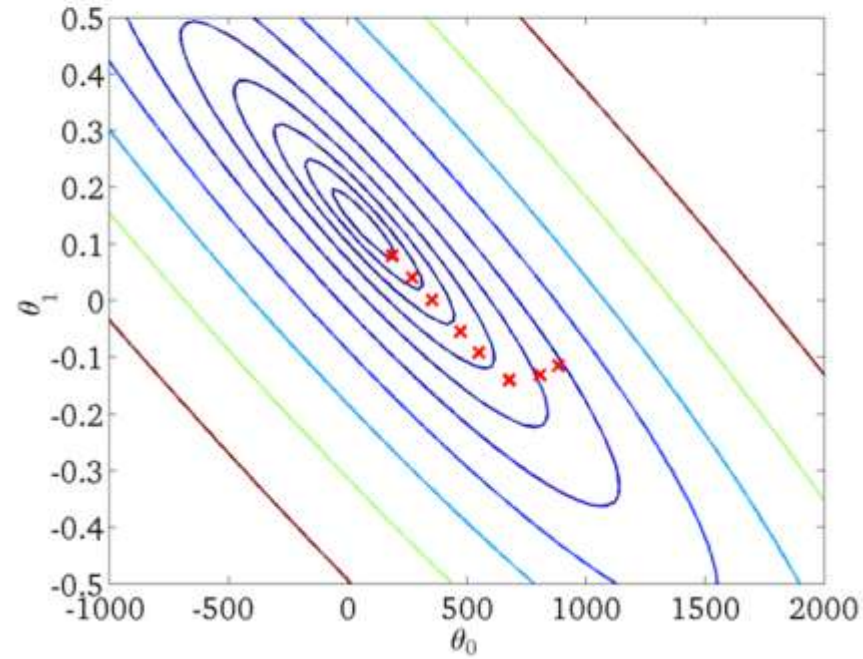
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

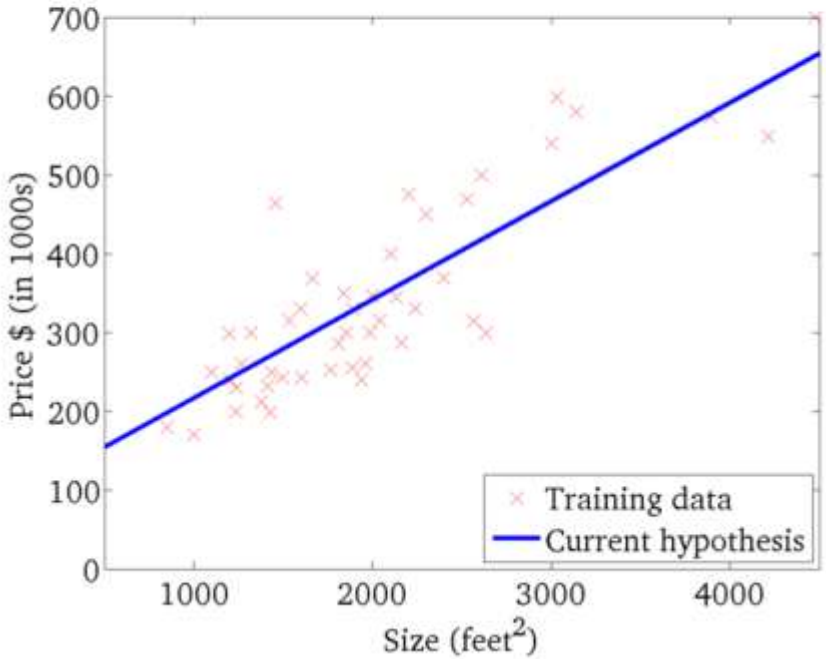


Source Credit : Slide by Andrew Ng

Gradient Descent

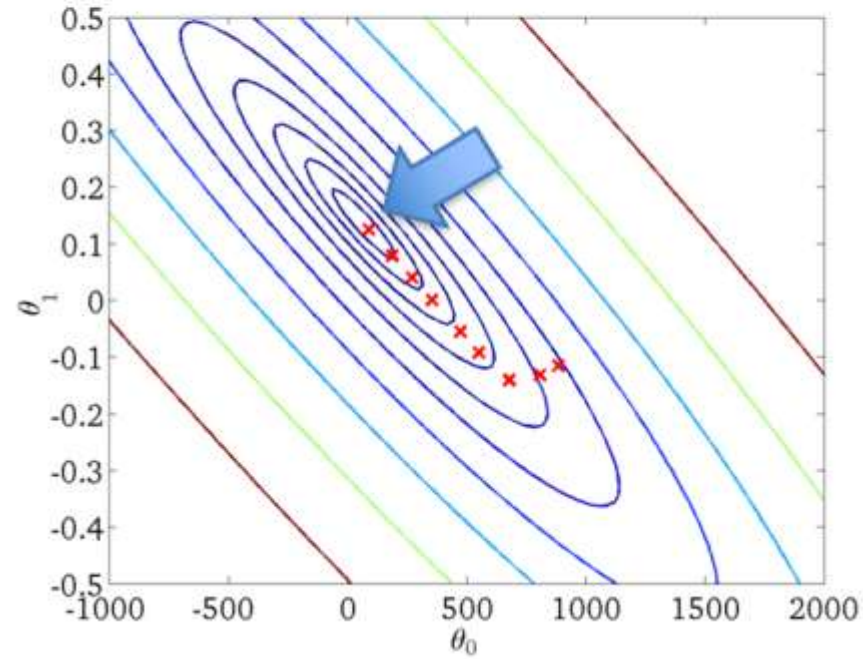
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



Source Credit : Slide by Andrew Ng

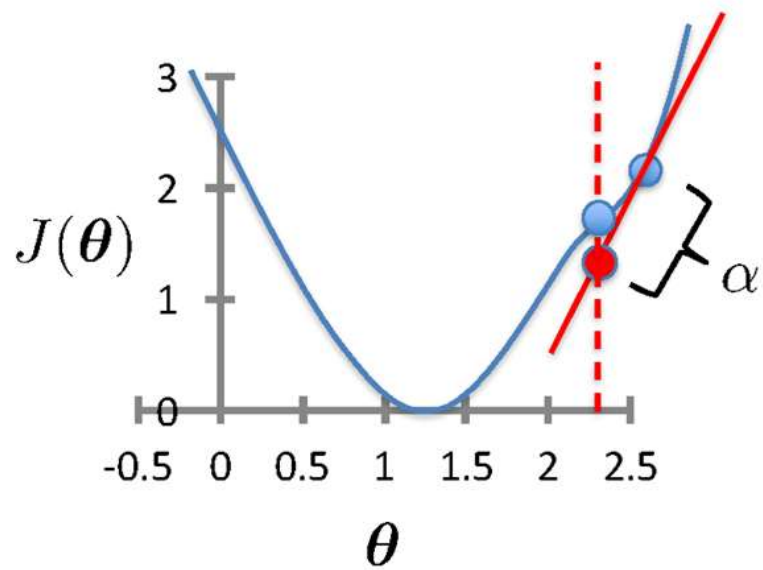
Gradient Descent

- Initialize $\boldsymbol{\theta}$
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta})$$

simultaneous update
for j = 0 ... d

learning rate (small)
e.g., α = 0.05



Gradient Descent

- Initialize $\boldsymbol{\theta}$
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta})$$

simultaneous update
for j = 0 ... d

For Linear Regression:

$$\begin{aligned}
\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta}) &= \frac{\partial}{\partial \theta_j} \frac{1}{2n} \sum_{i=1}^{n} \left(h_{\boldsymbol{\theta}} \left(\boldsymbol{x}^{(i)} \right) - y^{(i)} \right)^2 \\
&= \frac{\partial}{\partial \theta_j} \frac{1}{2n} \sum_{i=1}^{n} \left(\sum_{k=0}^{d} \theta_k x_k^{(i)} - y^{(i)} \right)^2 \\
&= \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{k=0}^{d} \theta_k x_k^{(i)} - y^{(i)} \right) \times \frac{\partial}{\partial \theta_j} \left(\sum_{k=0}^{d} \theta_k x_k^{(i)} - y^{(i)} \right) \\
&= \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{k=0}^{d} \theta_k x_k^{(i)} - y^{(i)} \right) x_j^{(i)}
\end{aligned}$$

Gradient Descent for Linear Regression

- Initialize $\boldsymbol{\theta}$
- Repeat until convergence

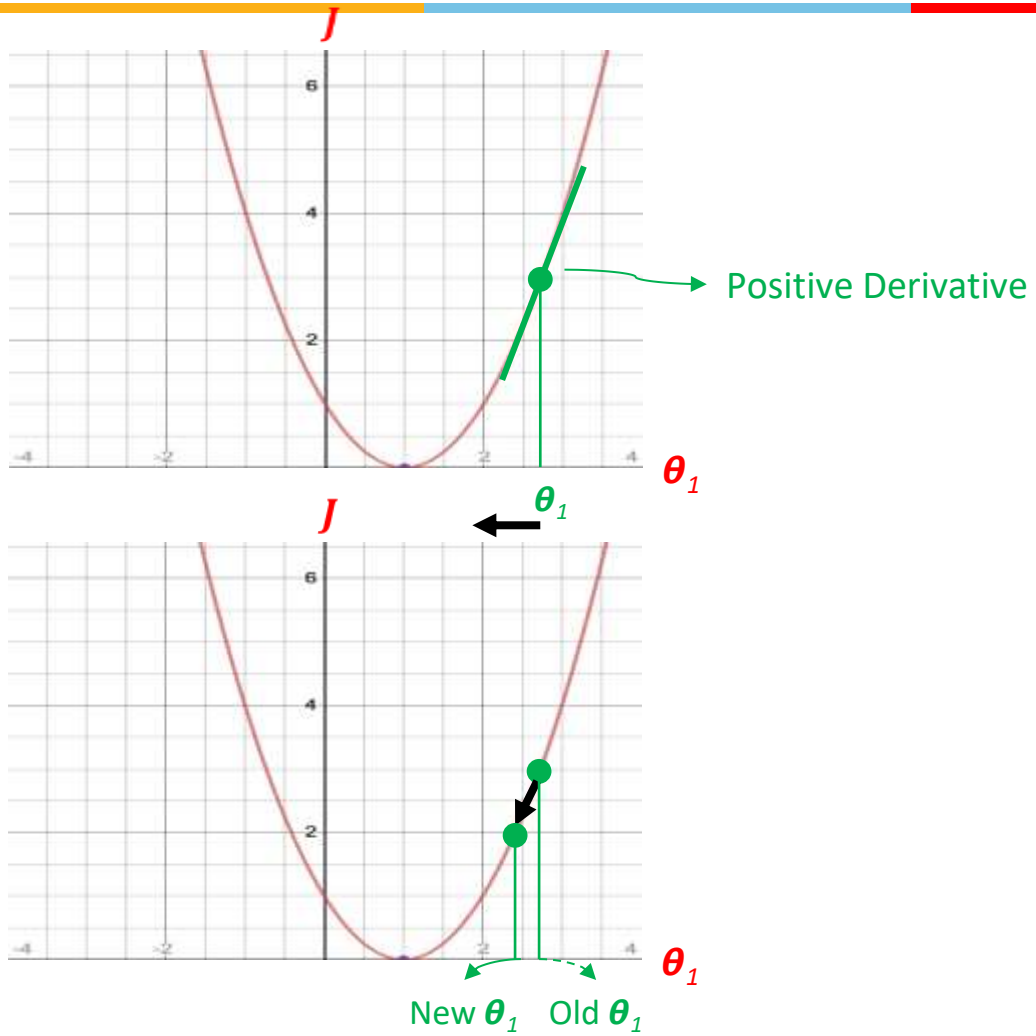
$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^{n} \left(h_{\boldsymbol{\theta}} \left(\boldsymbol{x}^{(i)} \right) - y^{(i)} \right) x_j^{(i)}$$

simultaneous update for j = 0 ... d

- To achieve simultaneous update
  - At the start of each GD iteration, compute $h_{\boldsymbol{\theta}} \left(\boldsymbol{x}^{(i)} \right)$
  - Use this stored value in the update step loop
- Assume convergence when $\|\boldsymbol{\theta}_{new} - \boldsymbol{\theta}_{old}\|_2 < \epsilon$

L_2 norm: $\|\boldsymbol{v}\|_2 = \sqrt{\sum_i v_i^2} = \sqrt{v_1^2 + v_2^2 + \ldots + v_{|v|}^2}$

Guarantee of Convergence



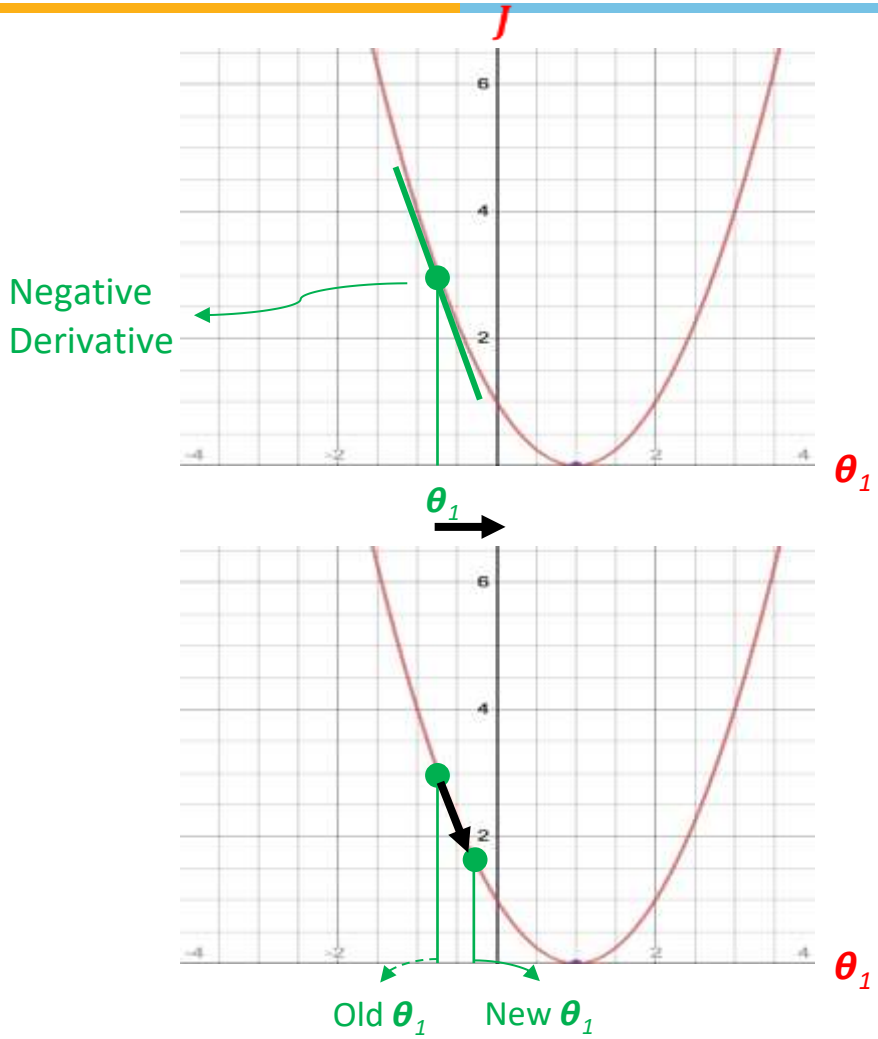
$$\theta_1 = \theta_1 - \alpha \frac{d\,J(\theta_1)}{d\,\theta_j}$$

$$= \theta_1 - \alpha\,(Positive\ Number)$$

Decrease θ_1 by a certain value

Source Credit: Prof. Mohammad Hammoud

Guarantee of Convergence



$$\theta_1 = \theta_1 - \alpha \frac{d\,J(\theta_1)}{d\,\theta_j}$$

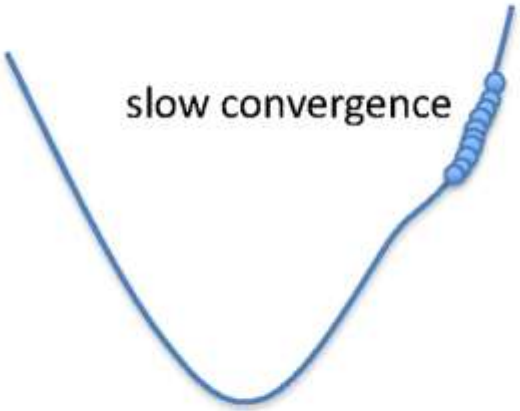
$$= \theta_1 - \alpha\,(Negative\ Number)$$

Increase θ_1 by a certain value

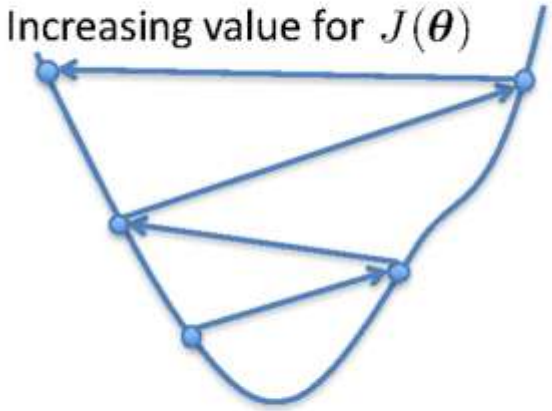
Source Credit: Prof. Mohammad Hammoud

Choosing Learning Rate

α too small



α too large



- May overshoot the minimum
- May fail to converge
- May even diverge

To see if gradient descent is working, print out $J(\theta)$ each iteration

- The value should decrease at each iteration
- If it doesn't, adjust α

Gradient Descent algorithm : Effect of Feature Scaling

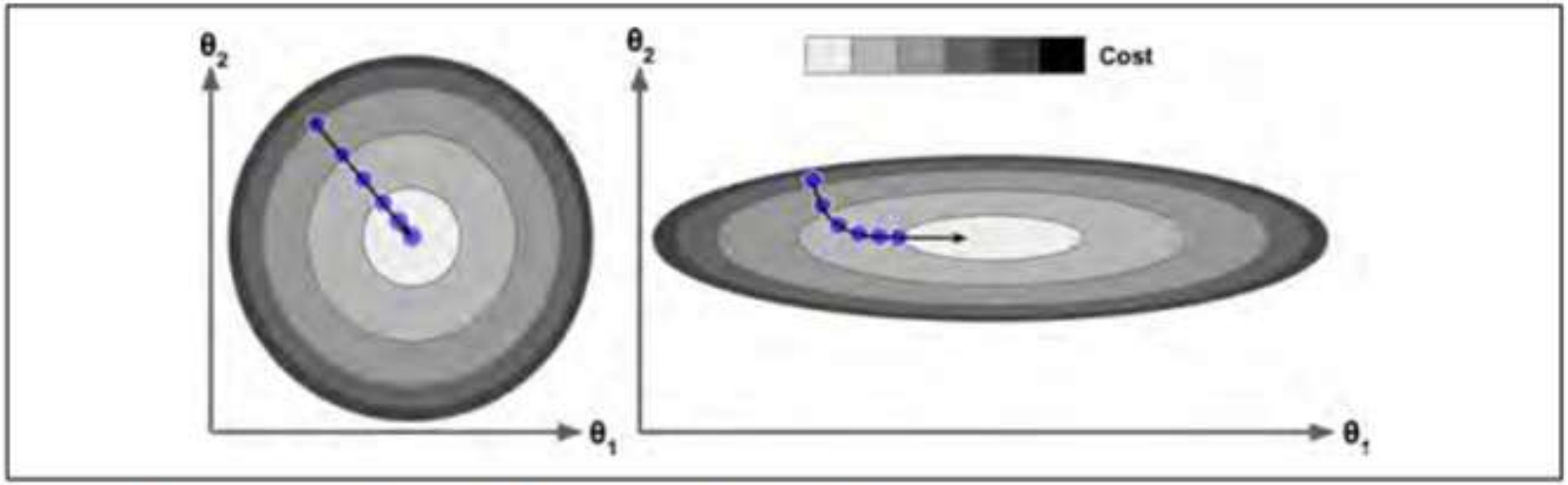


Figure 4-7. Gradient Descent with and without feature scaling

Gradient Descent algorithm : Effect of Learning Rate :

Problem Type 2 – Interpretation of Convergence or Effect of Hyper parameters

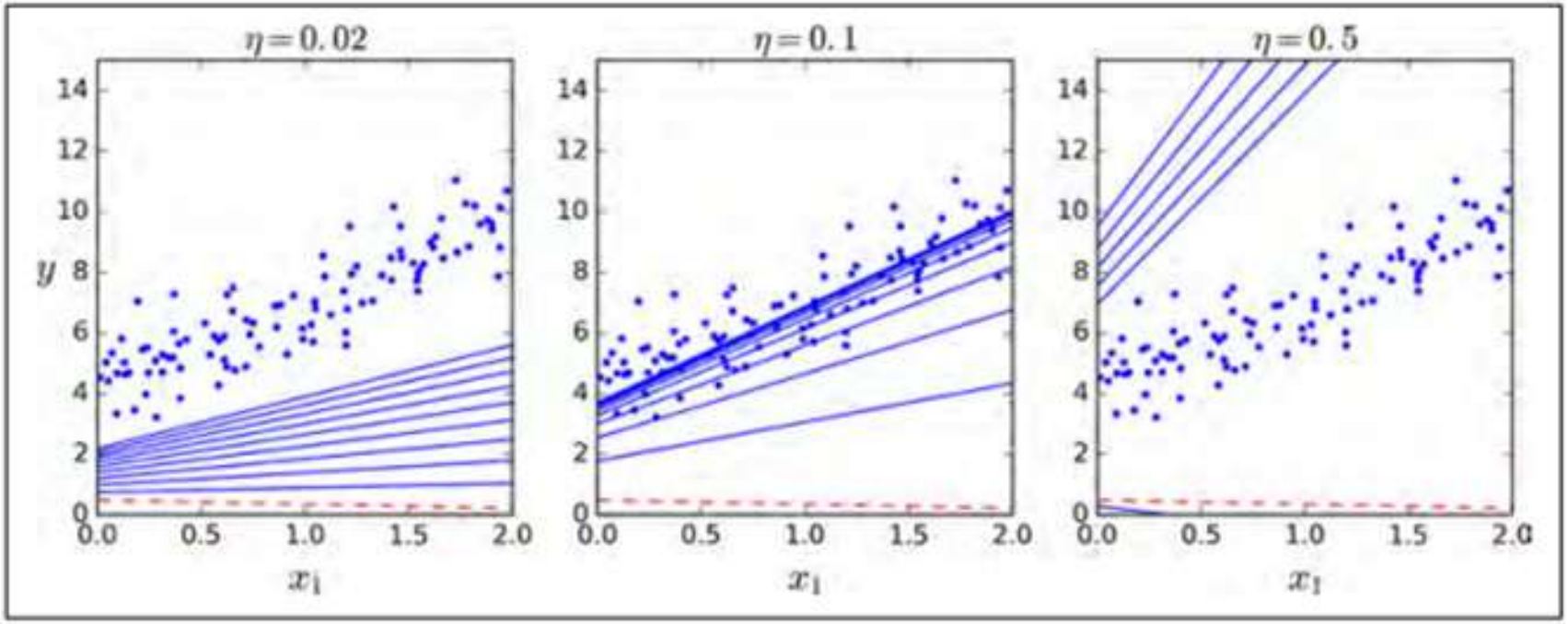


Figure 4-8. Gradient Descent with various learning rates

Fit a linear Regression Line :

Gradient Descent : Problem Type 3

Steps :

(Assuming : 'n' no.of.instances and two predictors {x1, x2} and linear regression)

1. Identification of the equations y = w0+w1X1+W2X2
2. Cost function & derivative
    1. W0` = w0 – 1/n * learning rate * (sum (w0+w1X1+W2X2 - y))
    2. W1` = w1 – 1/n * learning rate * (sum (w0+w1X1+W2X2 - y) * x1)
    3. W2` = w2 – 1/n * learning rate * (sum (w0+w1X1+W2X2 - y)*x2)
3. Apply the equations

Fit a linear Regression Line :

Gradient Descent

Fit a linear regression. Show only the first iteration of Gradient descent algorithm using learning rate of **0.02** for the following data , if the Relative Risk of Coronary Heart Disease is believed to be only linearly dependent on BMI as well as Diastolic Pressure. Assume the intercept of the regression model as **5** and the slope of independent variables as **-0.03 (negative)**.

Patient	Systolic Pressure mm Hg	Diastolic Pressure mm Hg	BMI	Waist Circumference Threshold cm	RR-CHD (Relative Risk of Coronary Heart Disease)
1	140	80	35	100	1.81
2	120	80	25	80	1.22
3	130	100	30	60	1.71

Steps :

1. Identification of the equations y = w0+w1X1+W2X2
2. Cost function & derivative
   1. W0` = w0 – 1/3 * learning rate * (sum (w0+w1X1+W2X2 - y))
   2. W1` = w1 – 1/3 * learning rate * (sum (w0+w1X1+W2X2 - y) * x1)
   3. W2` = w2 – 1/3 * learning rate * (sum (w0+w1X1+W2X2 - y)*x2)
3. Apply the equations

Fit a linear Regression Line :

Gradient Descent

Fit a linear regression. Show only the first iteration of Gradient descent algorithm using learning rate of **0.02** for the following data , if the Relative Risk of Coronary Heart Disease is believed to be only linearly dependent on BMI as well as Diastolic Pressure. Assume the intercept of the regression model as **5** and the slope of independent variables as **-0.03 (negative)**.

Patient	Systolic Pressure mm Hg	Diastolic Pressure mm Hg	BMI	Waist Circumference Threshold cm	RR-CHD (Relative Risk of Coronary Heart Disease)
1	140	80	35	100	1.81
2	120	80	25	80	1.22
3	130	100	30	60	1.71

Steps :

1. Identification of the equations: RR-CHD = 5 – 0.03 * BMI – 0.03 * DiastolicPressure
2. Cost function & derivative
   1. W0` = w0 – 1/3 * 0.02 * (sum (5-0.03BMI-0.03DiastolicPresure - RRCHD))
      = 5 – 1/3 * 0.02 * (sum (5-0.03BMI-0.03DiastolicPresure - RRCHD))
   2. W1` = w1 – 1/3 * 0.02 * (sum (5-0.03BMI-0.03DiastolicPresure - RRCHD) * BMI)
      = -0.03 – 1/3 * 0.02 * (sum (5-0.03BMI-0.03DiastolicPresure - RRCHD) * BMI)
   3. W2` = w2 – 1/3 * 0.02 * (sum (5-0.03BMI-0.03DiastolicPresure - RRCHD) * DiastolicPresure)
      = -0.03 – 1/3 * 0.02 * (sum (5-0.03BMI-0.03DiastolicPresure - RRCHD) * DiastolicPresure)
3. Apply the equations : Answer at the end of first iteration:
   W0 = 5.0016 , W1 = 0.0476, w2= 0.179

Closed Form Solution Vs. Gradient Descent

Gradient Descent	Closed Form Solution
• Requires multiple iterations • Need to choose α • Works well when n is large • Can support incremental learning	• Non-iterative • No need for α • Slow if n is large – Computing $(X^T X)^{-1}$ is roughly $O(n^3)$

Types of Gradient Descent Algorithms

Gradient Descent: Variants

- **Batch** gradient descent refers to calculating the derivative from all training data before calculating an update.
- **Minibatch** refers to calculating derivative of mini groups of training data before calculating an update.
- **Stochastic** gradient descent refers to calculating the derivative from each training data instance and calculating the update immediately

repeat until convergence {

$$\frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$

$$\theta_0 := \theta_0 - \alpha \boxed{\frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})}$$

$$\theta_1 := \theta_1 - \alpha \boxed{\frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)}) \cdot x^{(i)}}$$

$$\frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$

}

Update θ_0 and θ_1 simultaneously

$$temp0 := \theta_0 - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})$$

$$temp1 := \theta_1 - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})\, x^{(i)}$$

$$\theta_0 := temp0$$

$$\theta_1 := temp1$$

Gradient Descent: Variants

- **Batch** gradient descent refers to calculating the derivative from all training data before calculating an update.

Initialize the Parameters $(\theta_0^1, \theta_1^1, \ldots..)$
K=1
Repeat until Convergence {

$$\theta_0^{k+1} = \theta_0^k - \frac{\alpha}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})$$

$$\theta_1^{k+1} = \theta_1^k - \frac{\alpha}{m} \sum_{i=1}^{m} ((h_\theta(x^{(i)}) - y^{(i)}) * x_1^{(i)})$$

$$\theta_2^{k+1} = \theta_2^k - \frac{\alpha}{m} \sum_{i=1}^{m} ((h_\theta(x^{(i)}) - y^{(i)}) * x_2^{(i)})$$

$$\ldots\ldots$$

K=k+1
}
return $(\theta_0^1, \theta_1^1, \ldots..)$

Gradient Descent: Variants

- **Minibatch** refers to calculating derivative of mini groups of training data before calculating an update.

Divide the training instances into "N" batches each of size "m"
Initialize the Parameters $(\theta_0^1, \theta_1^1, \ldots..)$
K=1
Repeat until Convergence {
Repeat for every batch in 1 : N , each with 'm' instances {

$$\theta_0^{k+1} = \theta_0^k - \frac{\alpha}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})$$

$$\theta_1^{k+1} = \theta_1^k - \frac{\alpha}{m} \sum_{i=1}^{m} ((h_\theta(x^{(i)}) - y^{(i)}) * x_1^{(i)})$$

$$\theta_2^{k+1} = \theta_2^k - \frac{\alpha}{m} \sum_{i=1}^{m} ((h_\theta(x^{(i)}) - y^{(i)}) * x_2^{(i)})$$

……
K=k+1
}
}
return $(\theta_0^1, \theta_1^1, \ldots..)$

Gradient Descent: Variants

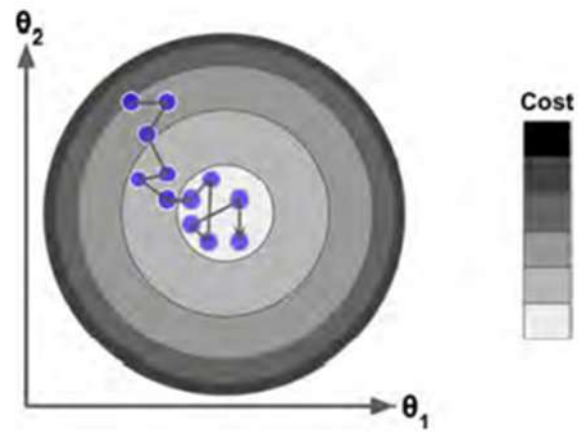
- **Stochastic gradient** descent refers to calculating the derivative from each training data instance and calculating the update immediately

Randomly shuffle training instances
Initialize the Parameters $(\theta_0^1, \theta_1^1, \ldots..)$
K=1
Repeat until Convergence {
Sample with replacement, only one random training instance "i" at a time

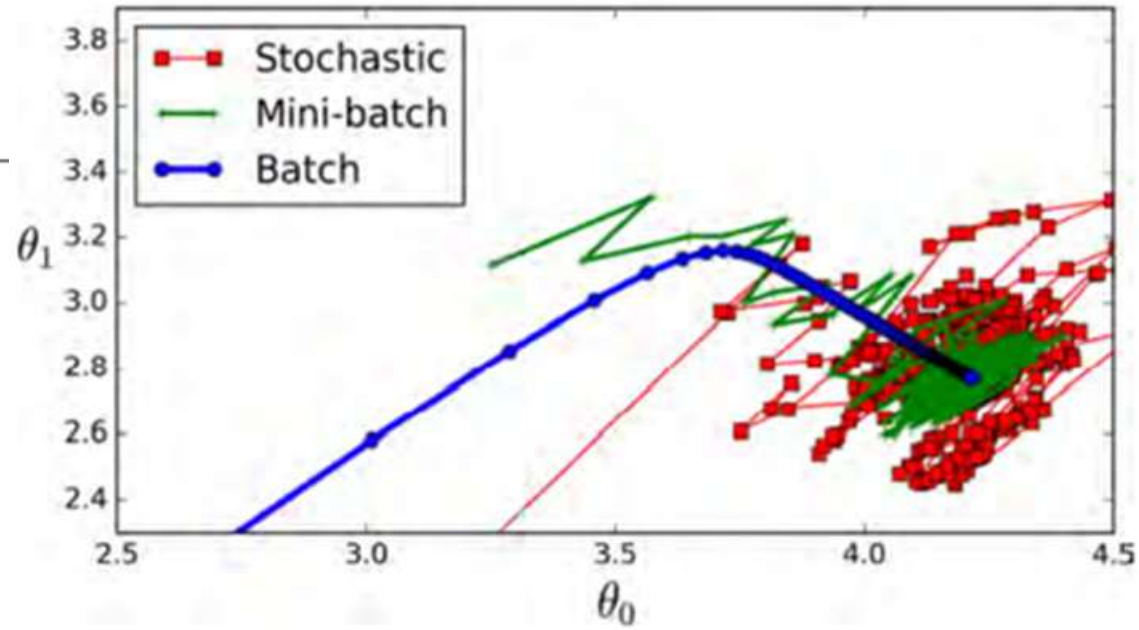
$$\theta_0^{k+1} = \theta_0^k - \alpha(h_\theta(x^{(i)}) - y^{(i)})$$
$$\theta_1^{k+1} = \theta_1^k - \alpha(h_\theta(x^{(i)}) - y^{(i)}) * x_1^{(i)}$$
$$\theta_2^{k+1} = \theta_2^k - \alpha(h_\theta(x^{(i)}) - y^{(i)}) * x_2^{(i)}$$

……
K=k+1
}
}
return $(\theta_0^1, \theta_1^1, \ldots..)$

Gradient Descent: Variants - Effect on Convergence



Stochastic Gradient Descent



1. Gradient Descent paths in parameter space

Evaluation Metrics

Evaluation of Linear Regression Model

Mileage (in kmpl)	*Car Price (in cr)*
9.8	10.48
9.12	1.75
9.5	6.95
10	2.51
….	…..

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}|$$

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y})^2$$

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y})^2}$$

Unseen Data	
Mileage (in kmpl)	***Car Price (in cr)***
7.5	9.25
10	6.5
….	…..

$$R^2 = 1 - \frac{SS_{residual}}{SS_{Total}}$$

Model 1

CarPrice = 8.5 + 0.5 Mileage – 1.5 Mileage^2

Model 2

CarPrice = -5.5 + 1.5 Mileage

Evaluation of Linear Regression Model

R-squared

Mileage (in kmpl)	*Car Price (in cr)*
9.8	10.48
9.12	1.75
9.5	6.95
10	2.51
….	…..

variation in 'y' that is explained by a regression model

$$explained\ variation = \widehat{y} - \overline{y}$$

variation in 'y' that is not captured/explained by a regression model

$$unexplained\ variation = y - \widehat{y}$$

$$total\ variation = (y - \widehat{y}) + (\widehat{y} - \overline{y}) = (y - \overline{y})$$

Car Price (in cr)
10.48
1.75
6.95
2.51
Mean Y

$$R^2 = 1 - \frac{SS_{residual}}{SS_{Total}}$$

$$SS_{explained} = \sum_{i=1}^{n} (\widehat{y}_i - \overline{y})^2$$

$$SS_{residual} = \sum_{i=1}^{n} (y_i - \widehat{y}_i)^2$$

$$SS_{Total} = \sum_{i=1}^{n} (y_i - \overline{y})^2$$

$where:$

$SS_{explained}$ = *explained variation sum of squares*

$SS_{residual}$ = *unexplained variation sum of squares*

SS_{Total} = *total variation sum of squares*

Model 1

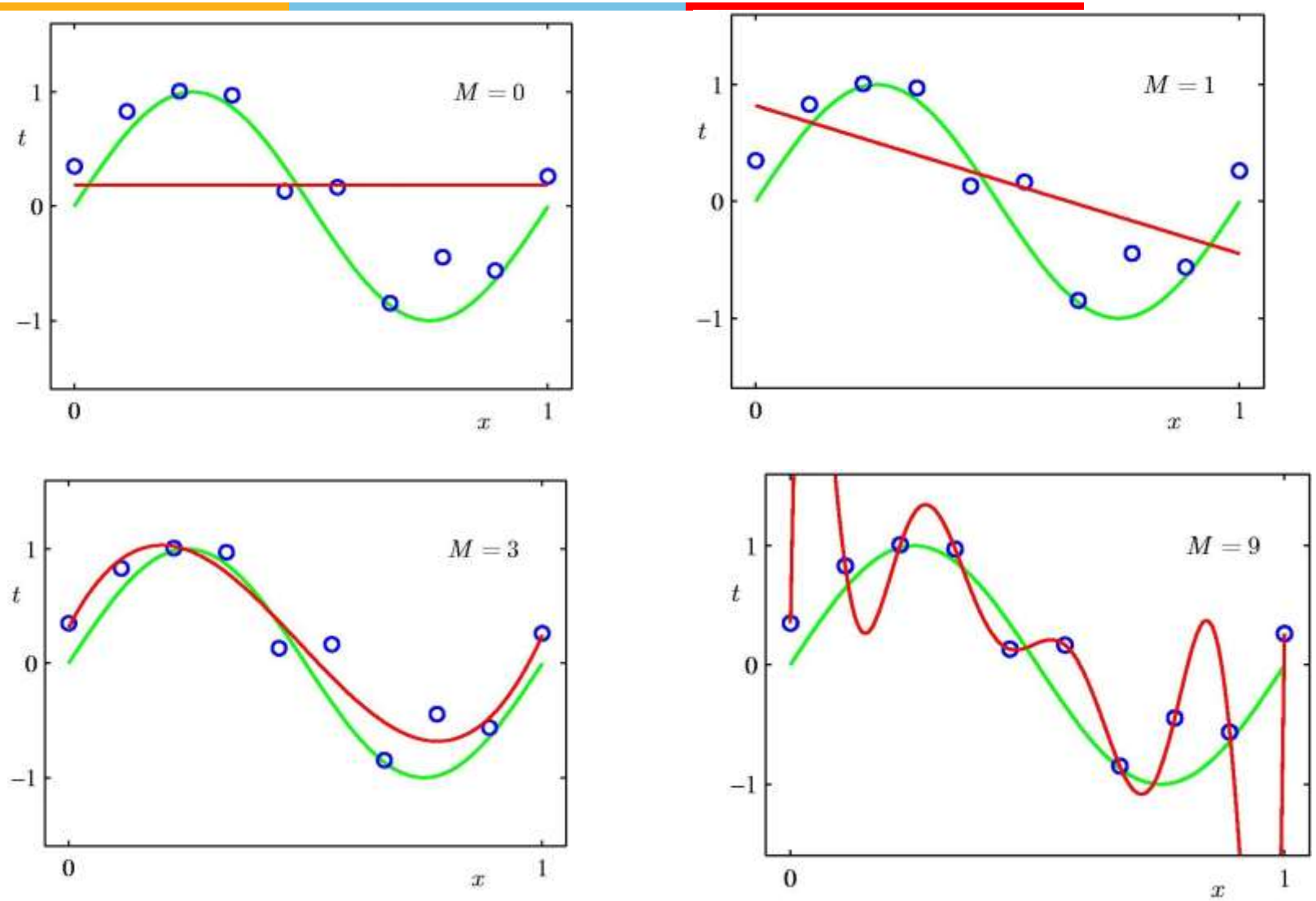
CarPrice = 8.5 + 0.5 Mileage – 1.5 Mileage^2

- **variation that is explained by a regression model**
- **measures the goodness of fit of a regression model**

Linear Basis Models

What if output is a non-linear function of input vector?

Polynomial Regression



Linear Basis Function Models

- The inputs **X** for linear regression can be:
  - Original quantitative inputs
  - Transformation of quantitative inputs
    - e.g. log, exp, square root, square, etc.
  - Polynomial transformation
    - example: $y = \beta_0 + \beta_1 \cdot x + \beta_2 \cdot x^2 + \beta_3 \cdot x^3$
  - Basis expansions
  - Dummy coding of categorical inputs
  - Interactions between variables
    - example: $x_3 = x_1 \cdot x_2$

This allows use of linear regression techniques to fit non-linear datasets.

Input X	Output Y
exp(2)	....
exp(4)	
exp(6.3)	
exp(9.2)	

X No.of.Years of Experience (in Years)	X^2	Y Salary Of the Employee (in Lakhs)
1	1	2
2	4	3
3	9	4
4	16	5
5	25	6

X1 = Graduate	X2 = PostGraduate	X3 = Others	Y Salary Of the Employee
0	0	1	2
1	0	0	3
0	0	1	4
0	1	0	5
1	0	0	6

Linear Basis Function Models

X No.of.Years of Experience (in Years)	X^2	Y Salary Of the Employee (in Lakhs)
1	1	2
2	4	3
3	9	4
4	16	5
5	25	6

Example: an M-th order polynomial function of one dimensional feature x :

$$y(x, \mathbf{w}) = w_0 + \sum_{j=1}^{M} w_j x^j$$

where x^j = j-th power of x

$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \ldots + w_M x^M = \sum_{j=0}^{M} w_j x^j$$

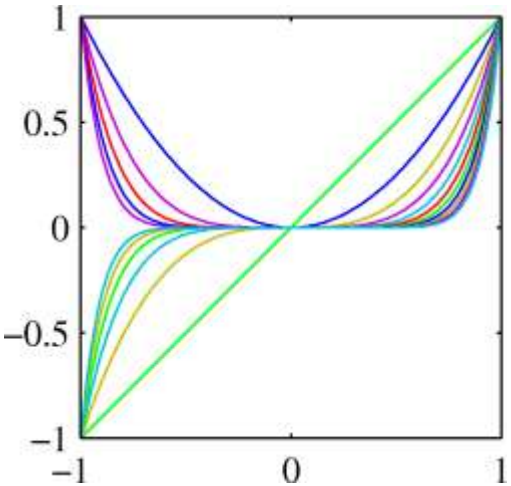
$$y(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^{M-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^{\mathrm{T}} \boldsymbol{\phi}(\mathbf{x})$$

$\Phi_j(x)$ are known as *basis functions*. Typically, $\Phi_0(x) = 1$, so that w_0 acts as a bias.

In the simplest case, we use linear basis functions : $\Phi_d(x) = x_d$.

They are called linear models because this function is linear in **w**.

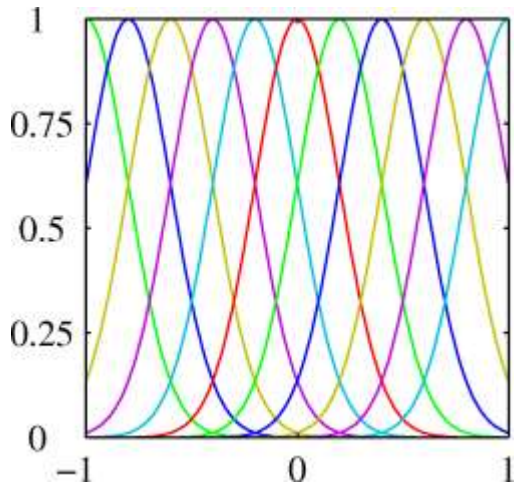
Linear Basis Function Models - Examples



Polynomial basis functions:

$$\phi_j(x) = x^j.$$

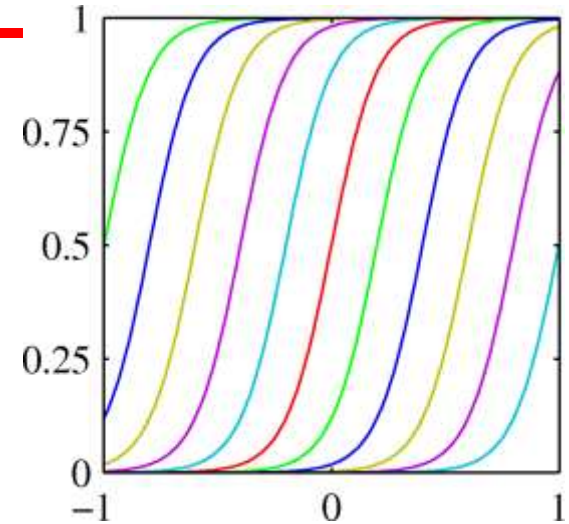
These are global; a small change in x affect all basis functions.



Gaussian basis functions:

$$\phi_j(x) = \exp\left\{-\frac{(x-\mu_j)^2}{2s^2}\right\}$$

These are local; a small change in x only affect nearby basis functions. μ_j and s control location and scale (width).



Sigmoidal basis functions:

where $\phi_j(x) = \sigma\left(\frac{x-\mu_j}{s}\right)$

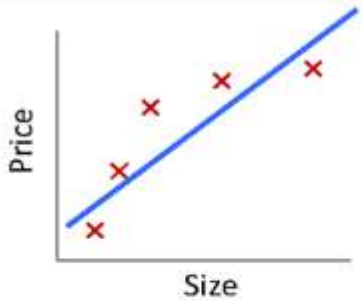
$$\sigma(a) = \frac{1}{1+\exp(-a)}.$$

Also these are local; a small change in x only affect nearby basis functions. μ_j and s control location and scale (slope).

Notion of Bias - Variance

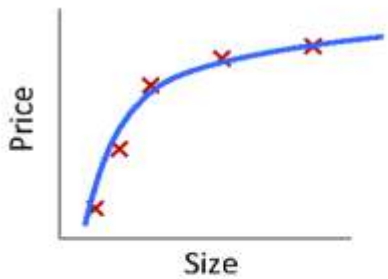
BITS Pilani

Quality of Fit



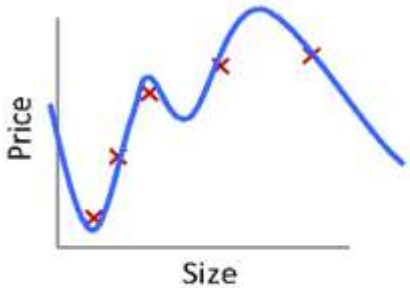
$\theta_0 + \theta_1 x$

Underfitting
(high bias)



$\theta_0 + \theta_1 x + \theta_2 x^2$

Correct fit



$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

Overfitting
(high variance)

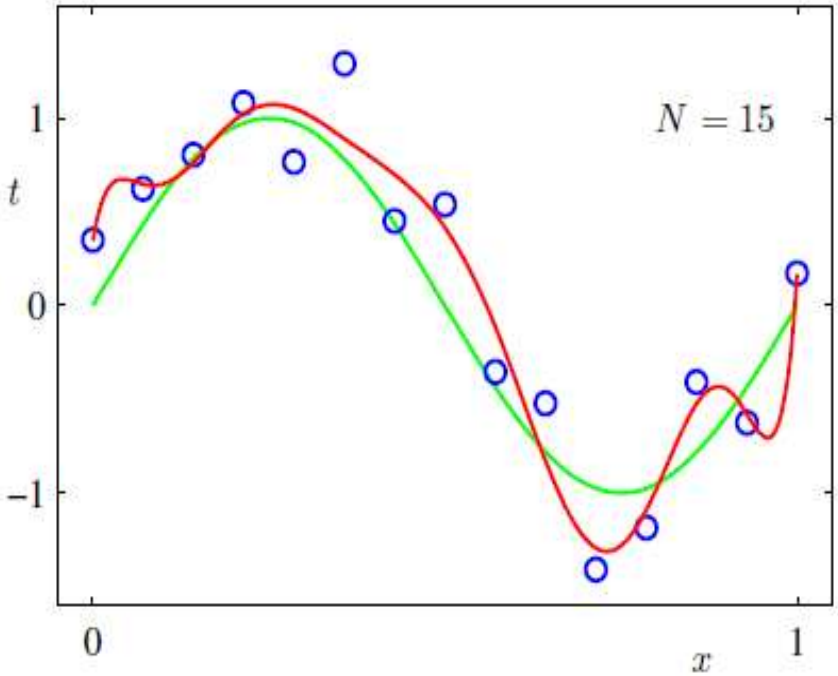
Overfitting:

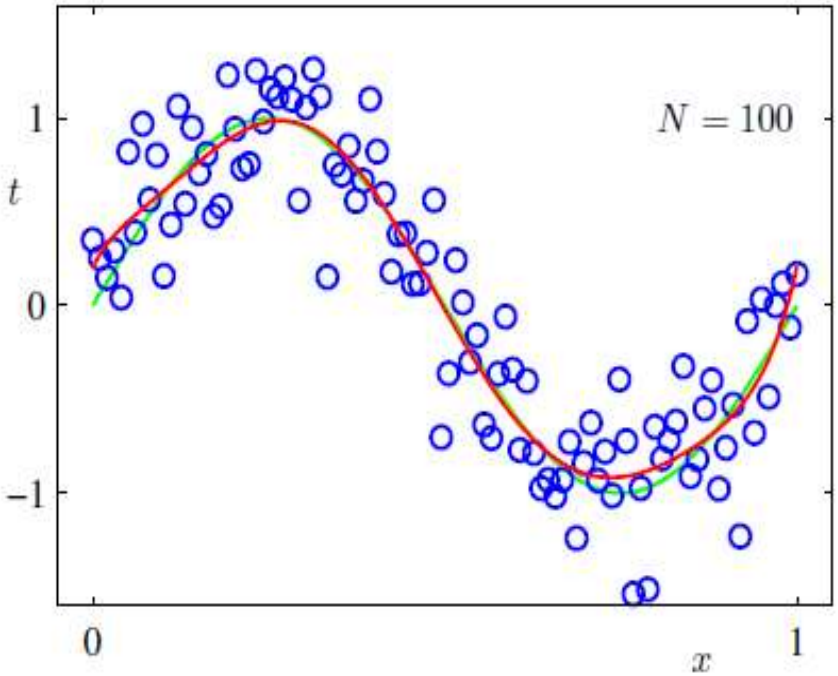
- The learned hypothesis may fit the training set very well ($J(\boldsymbol{\theta}) \approx 0$)
- ...but fails to generalize to new examples

Based on example by Andrew Ng

Handling Overfitting – Way 1

Increase in Size of the data set reduces the over-fitting problem

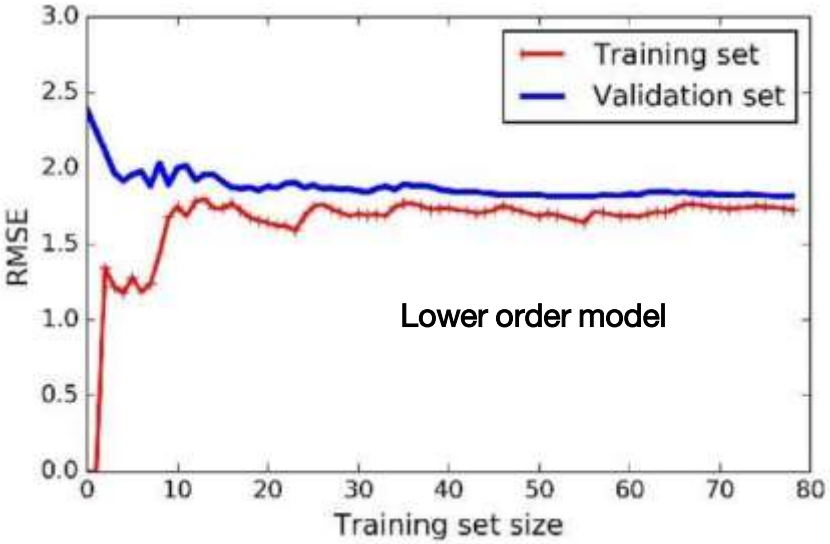


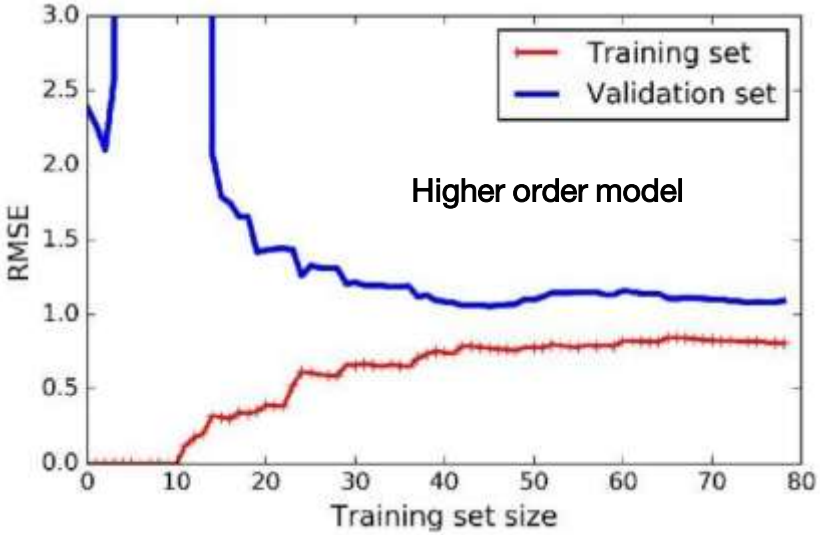


Effect of Training Size on Over fitting

Problem Type 4 : Interpretation of the Model Fit

- Size of training dataset needs to be large to prevent when higher order model is used.

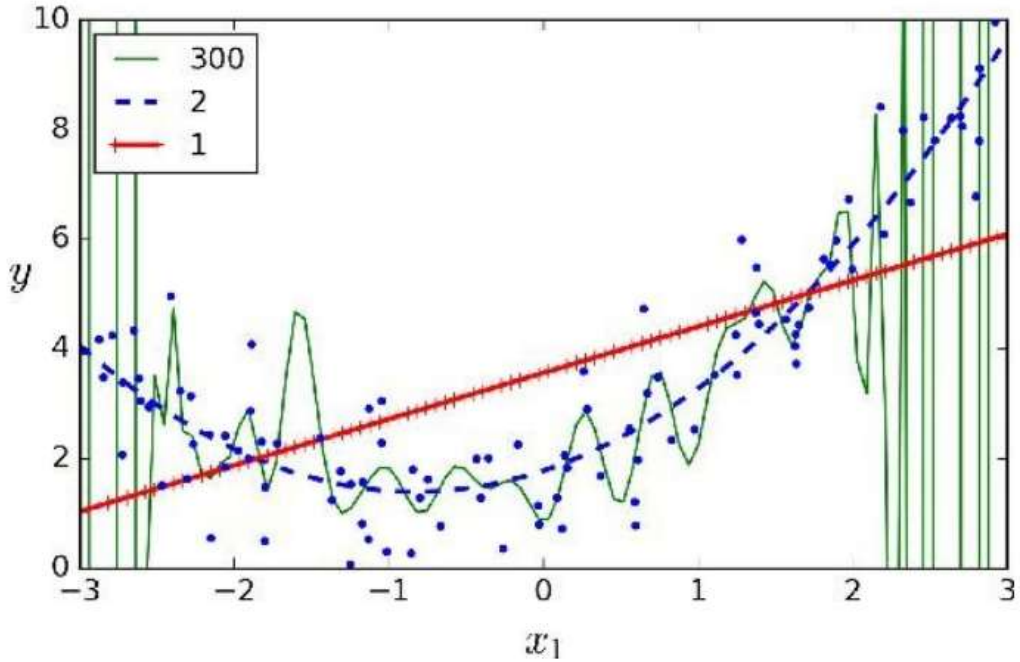




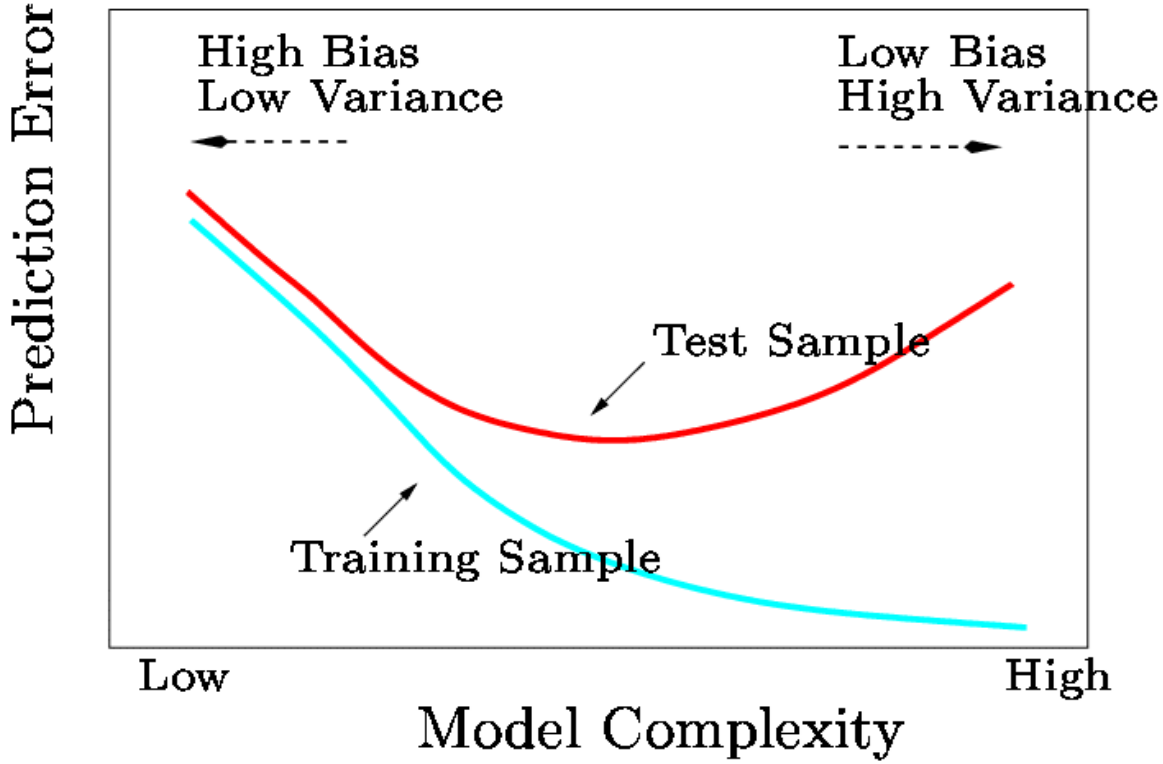
Polynomial Fitting can lead to Overfitting

Handling Overfitting – Way 2 – Reduce the complexity of the model

- Underlying target function is quadratic
- Linear model results in under fitting with large bias
- Polynomial of order 300 results in a large variance

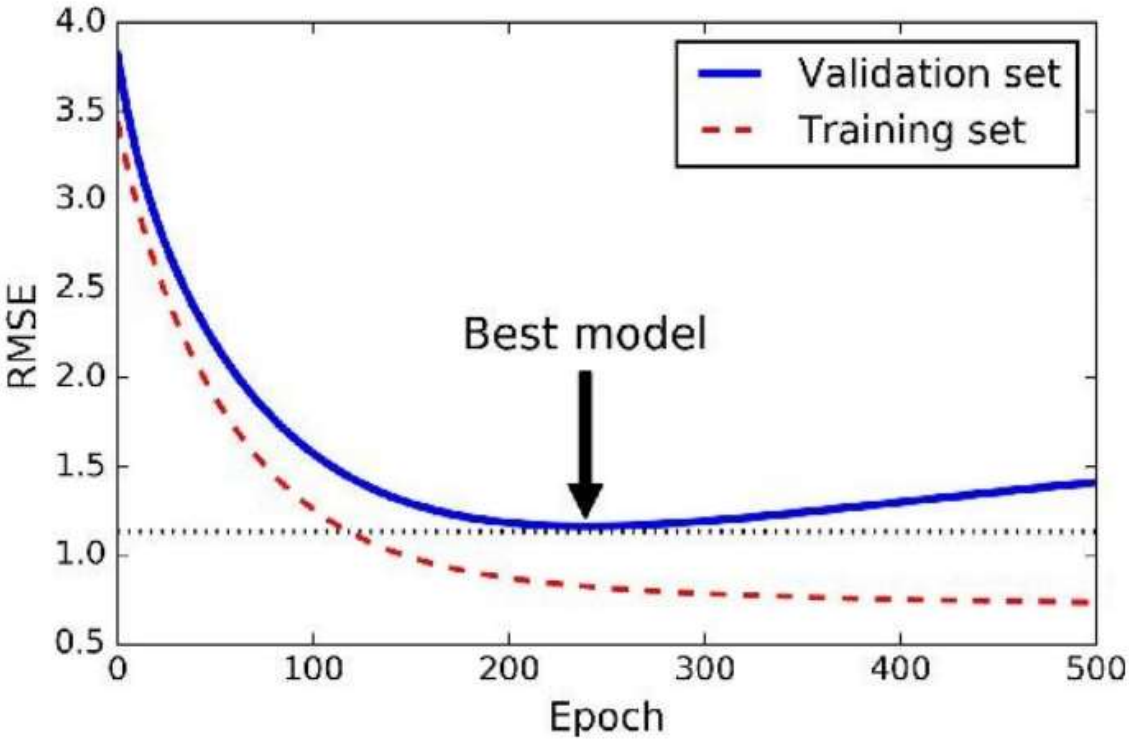


How to experiment on Model Complexity ?



Handling Overfitting – Way 3

Early Stop the Training



Regularization

(*This notion is common for both Linear Regression – Module 3 and Logistic Regression – Module 4)

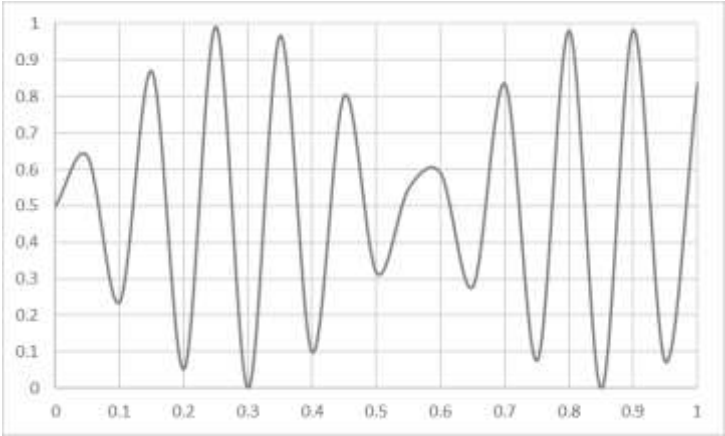
Overfitting vs Underfitting

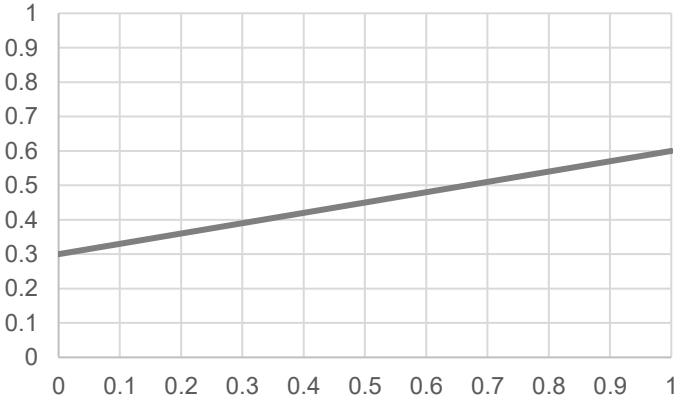
Overfitting

- Fitting the data too well
  - Features are noisy / uncorrelated to concept

Underfitting

- Learning too little of the true concept
  - Features don't capture concept
  - Too much bias in model





Regularization

- A method for automatically controlling the complexity of the learned hypothesis
- **Idea**: penalize for large values of θ_j
  - Can incorporate into the cost function
  - Works well when we have a lot of features, each that contributes a bit to predicting the label
- Can also address overfitting by eliminating features (either manually or via model selection)

Regularization

- Linear regression objective function

$$J(\boldsymbol{\theta}) = \underbrace{\frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2}_{\text{model fit to data}} + \underbrace{\frac{\lambda}{2}\sum_{j=1}^{d}\theta_j^2}_{\text{regularization}}$$

  - λ is the regularization parameter ($\lambda \geq 0$)
  - No regularization on θ_0!

Understanding Regularization

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2 + \frac{\lambda}{2}\sum_{j=1}^{d}\theta_j^2$$

- Note that $\sum_{j=1}^{d}\theta_j^2 = \|\boldsymbol{\theta}_{1:d}\|_2^2$
  - This is the magnitude of the feature coefficient vector!

- We can also think of this as:

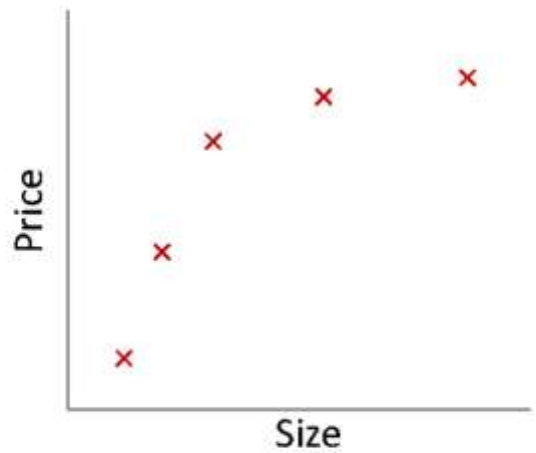
$$\sum_{j=1}^{d}(\theta_j - 0)^2 = \|\boldsymbol{\theta}_{1:d} - \vec{\mathbf{0}}\|_2^2$$

  - L_2 regularization pulls coefficients toward 0

Understanding Regularization

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2 + \frac{\lambda}{2}\sum_{j=1}^{d}\theta_j^2$$

- What happens if we set λ to be huge (e.g., 10^{10})?



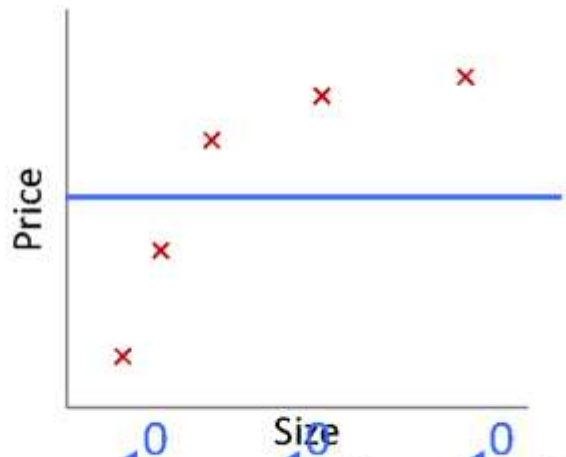
$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

Based on example by Andrew Ng

Understanding Regularization

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2 + \frac{\lambda}{2}\sum_{j=1}^{d}\theta_j^2$$

- What happens if we set λ to be huge (e.g., 10^{10})?



$$\theta_0 + \cancelto{0}{\theta_1}x + \cancelto{0}{\theta_2}x^2 + \cancelto{0}{\theta_3}x^3 + \cancelto{0}{\theta_4}x^4$$

Based on example by Andrew Ng

Regularization

Ridge Regression / Tikhonov regularization

- Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2 + \frac{\lambda}{2}\sum_{j=1}^{d}\theta_j^2$$

- Fit by solving $\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$

- Gradient update:

$\frac{\partial}{\partial\theta_0}J(\theta)$

$$\theta_0 \leftarrow \theta_0 - \alpha\frac{1}{n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)$$

$\frac{\partial}{\partial\theta_j}J(\theta)$

$$\theta_j \leftarrow \theta_j - \alpha\frac{1}{n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)x_j^{(i)} \underbrace{- \lambda\theta_j}_{\text{regularization}}$$

- We can rewrite the gradient step as:

$$\theta_j \leftarrow \theta_j\left(1 - \alpha\lambda\right) - \alpha\frac{1}{n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)x_j^{(i)}$$

Regularization

Lasso Regression (Least Absolute Shrinkage and Selection Operator Regression)

- Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2 + \lambda\sum_{j=1}^{d}|\theta_j|$$

- Fit by solving $\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$

- Gradient update:

$\frac{\partial}{\partial\theta_0}J(\theta)$

$$\theta_0 \leftarrow \theta_0 - \alpha\frac{1}{n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)$$

$\frac{\partial}{\partial\theta_j}J(\theta)$

$$\theta_j = \theta_j - \frac{\alpha}{n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)\boldsymbol{x}_j^{(i)} - \alpha\lambda\,\mathrm{sign}(\theta_j)$$

regularization

$$\text{where} \quad \mathrm{sign}\left(\theta_i\right) = \begin{cases} -1 & \text{if } \theta_i < 0 \\ 0 & \text{if } \theta_i = 0 \\ +1 & \text{if } \theta_i > 0 \end{cases}$$

Regularization

Elastic Net

- Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2 + r\,\frac{\lambda}{2}\sum_{j=1}^{d}\theta_j^2 + \frac{(1-r)}{2}\,\lambda\sum_{j=1}^{d}|\theta_j|$$

Control the regularization using **the Mix Ration "r":**
When,
r = 0, Elastic Net is equivalent to Ridge Regression,
r = 1, it is equivalent to Lasso Regression

```
from sklearn.linear_model import ElasticNet
ElasticNet(alpha=0.1, l1_ratio=0.5)
```

How to choose the right Regularization?

Common Usage & Observation

- L1 regularization has the ability to set some coefficients to 0 exactly leading to a *sparse* model
- L1 regularization helps in feature selection by eliminating the features that are not important
- **L1 cannot be used efficiently in gradient-based approaches since it is not-differentiable unlike L2**
- L2 will in general lead to small magnitudes of weights but not exactly 0.
- Elastic net – Prefer in Highly correlated features

Numerical Exercise – For Student Practice

Fit a linear regression. Show only the first iteration of Gradient descent algorithm using learning rate of **0.02** for the following data , if the Relative Risk of Coronary Heart Disease is believed to be only linearly dependent on BMI as well as Diastolic Pressure. Assume the intercept of the regression model as **5** and the slope of independent variables as **-0.03 (negative)**.

Patient	Systolic Pressure mm Hg	Diastolic Pressure mm Hg	BMI	Waist Circumference Threshold cm	RR-CHD (Relative Risk of Coronary Heart Disease)
1	140	80	35	100	1.81
2	120	80	25	80	1.22
3	130	100	30	60	1.71

Apply a regularization on the same problem with regularization constant 5 and apply GD for 2 iterations & interpret the results. Try both ridge regression as well as lasso regression. Below equation is changed only for ridge regression. Students must apply appropriate GD update equation changes for this problem.

Steps :

1. Identification of the equations y = w0+w1X1+W2X2
2. Cost function & derivative
   1. W0` = w0 – 1/3 * learning rate * (sum (w0+w1X1+W2X2 - y))
   2. W1` = w1* (1- learning rate * regularization constant) – 1/3 * learning rate * (sum (w0+w1X1+W2X2 - y) * x1)
   3. W2` = w2 * (1- learning rate * regularization constant) – 1/3 * learning rate * (sum (w0+w1X1+W2X2 - y)*x2)
3. Apply the equations

Python Lab Exercise – For Student Practice

Go to your virtual lab file under : Machine Learning ☐ LabCapsule 2 Linear _ Polynomial Regression ☐ 3 Polynomial Regression

Download ML_Lab 5 PolynomialRegression.ipynb

Try to change the degree to {2, 5, 7, 10} in the below function line : PolynomialFeatures(degree=3)

Observe and interpret on the performance of the training data

Note: Training vs Test Data split is not coded in this implementation. Below are the suggestions to experiment further:

- Add few hundreds of data (Use synthetic data generation python libraries available : Refer here)
- Split the data into 80% train vs 20% test set
- Built the polynomial model using above four different degree on training set
- For each of the model apply in the test set
- Find the MSE for both training data and separately for test data
- Plot the four experiment results in a plot and interpret the notion of overfit vs underfit
- Suggest which degree is a best fit .

References

- T1 - Chapter 1 – Machine Learning, Tom Mitchell
- Chapter 1, 2 – Introduction to Machine Learning, 2nd edition, Ethem Alpaydin
- R1 – Chapter #1, # 3,#4 (Christopher M. Bhisop, Pattern Recognition & Machine Learning) & Refresh your MFDS course basics

Thank you !